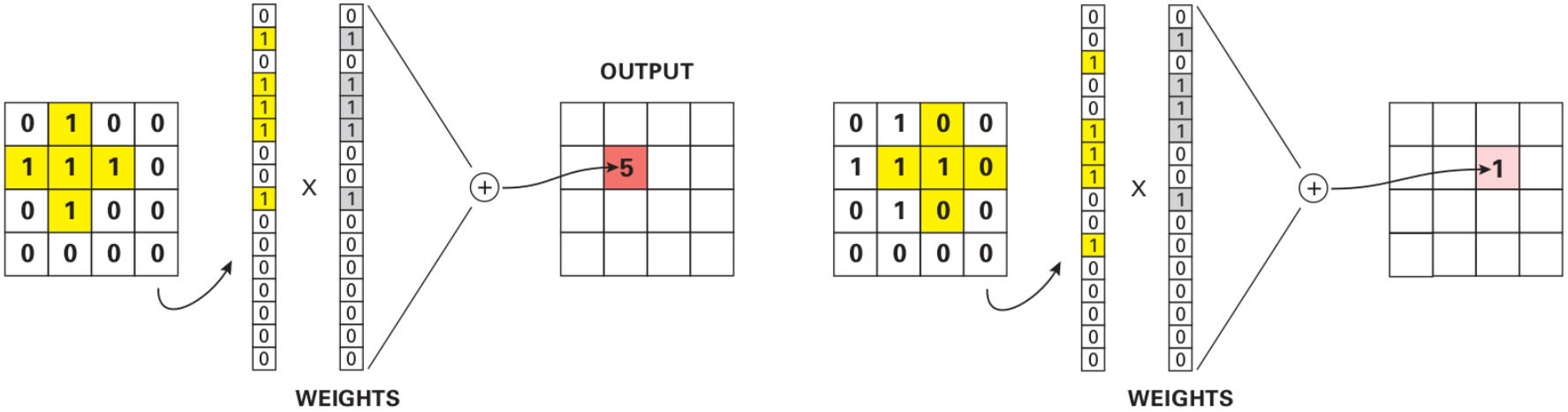
Part 2.3:

Convolutional Neural Networks (CNNs)

MLP doesn't Work Well for Images

- MLPs are not translation invariant
  - They may produce a strong response (e.g., 5) if an object is positioned on the left, but a weak response (e.g., 1) if the object shifts to the right



A bit of history:

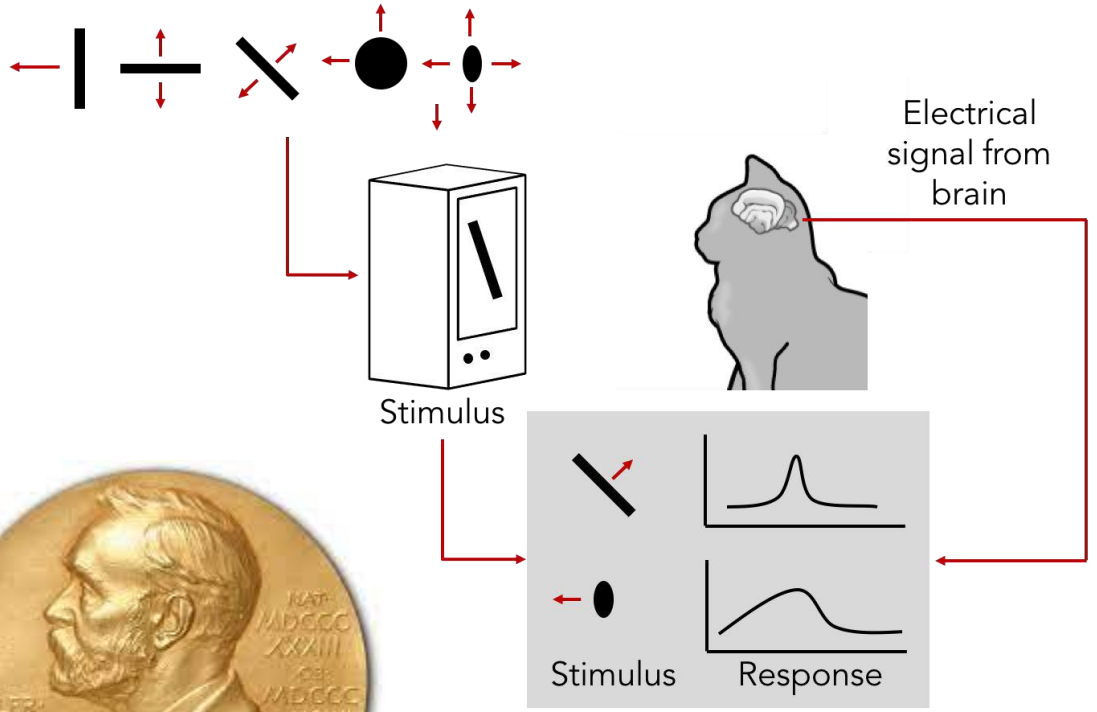
Hubel & Wiesel, 1959

RECEPTIVE FIELDS OF SINGLE NEURONES IN
THE CAT'S STRIATE CORTEX

1962

RECEPTIVE FIELDS, BINOCULAR INTERACTION
AND FUNCTIONAL ARCHITECTURE IN
THE CAT'S VISUAL CORTEX

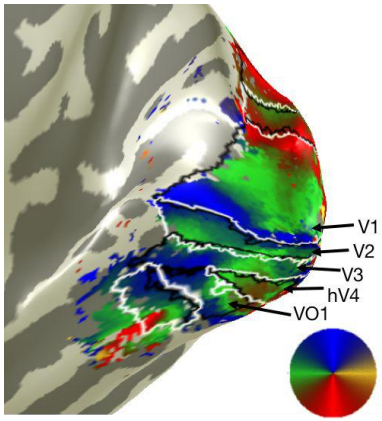




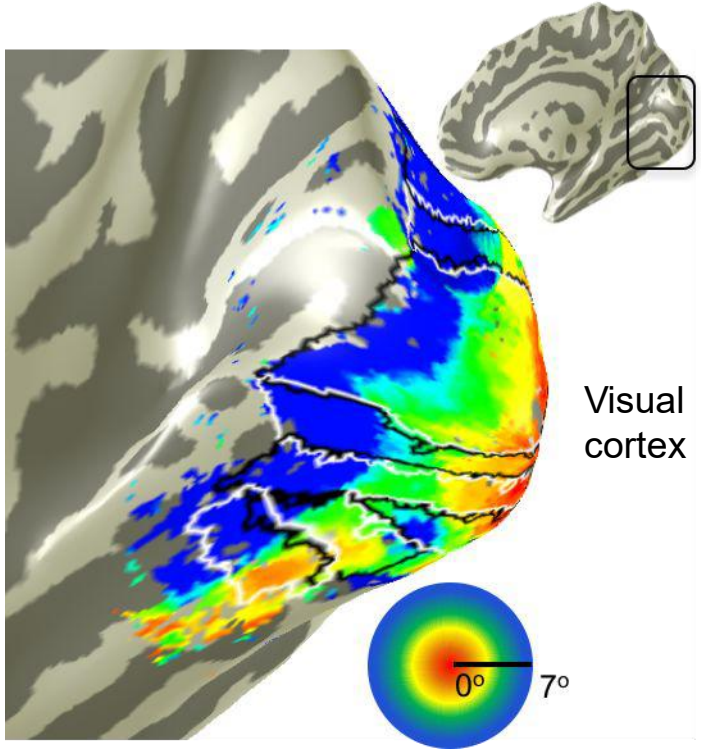
Cat image by CNX OpenStax is licensed under CC BY 4.0; changes made

A bit of history

Topographical mapping in the cortex:
nearby cells in cortex represent
nearby regions in the visual field

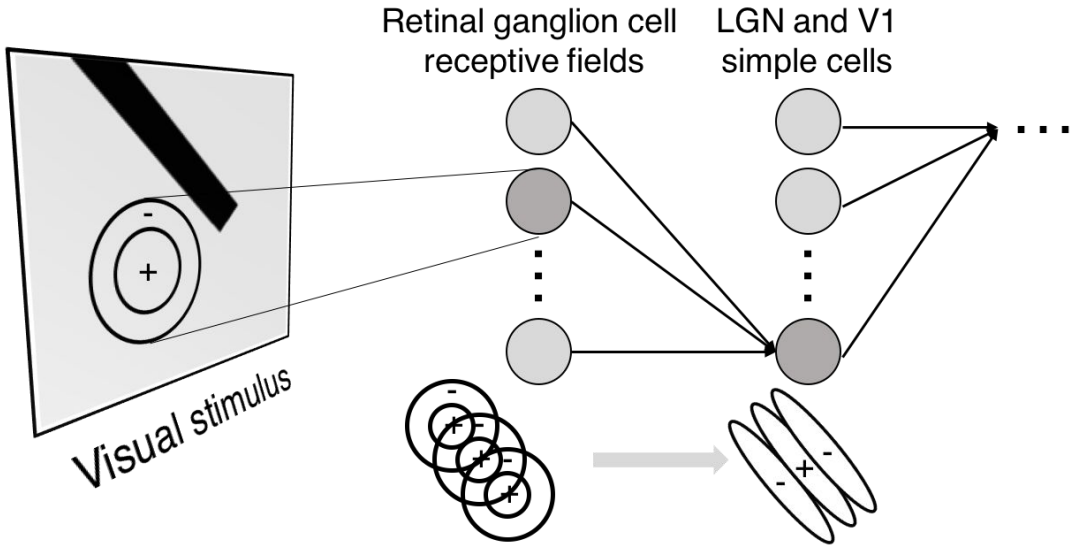


Human brain



Retinotopy images courtesy of Jesse Gomez in the Stanford Vision & Perception Neuroscience Lab.

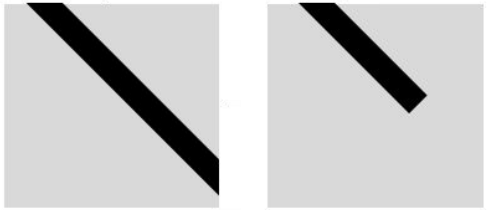
Hierarchical organization



Simple cells:
Response to light orientation

Complex cells:
Response to light orientation and movement

Hypercomplex cells:
response to movement with an end point

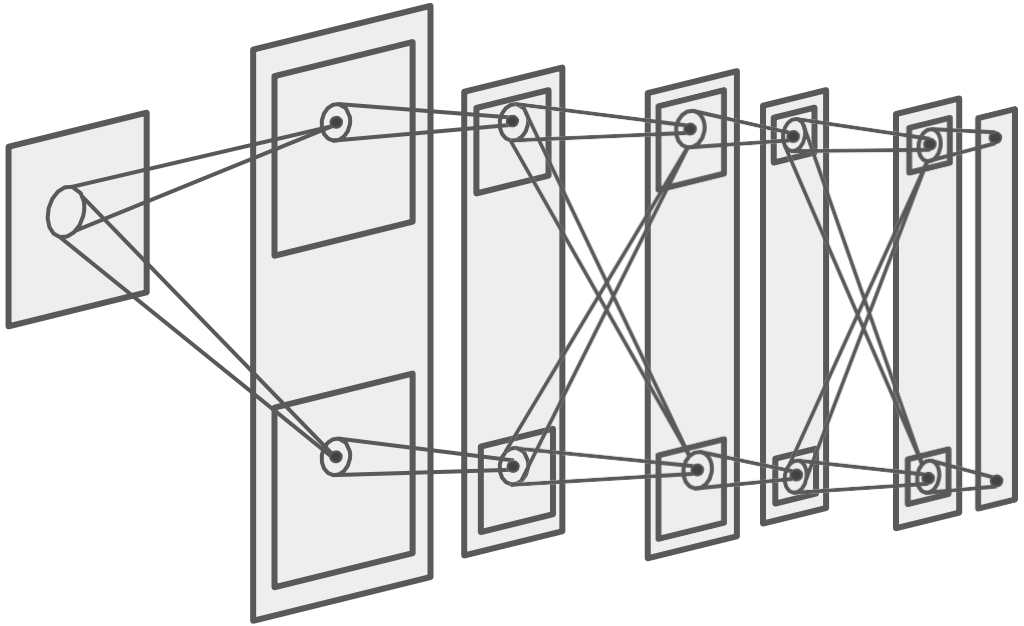


No response

Response (end point)

A bit of history:

Neocognitron *[1980]*

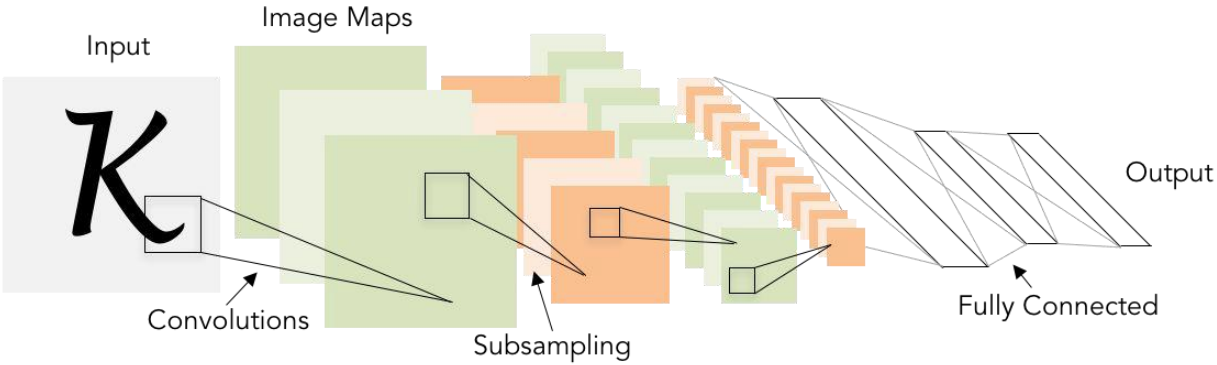


"sandwich" architecture (SCSCSC…)
simple cells: modifiable parameters
complex cells: perform pooling

A bit of history:

Gradient-based learning applied to document recognition

[LeCun, Bottou, Bengio, Haffner 1998]

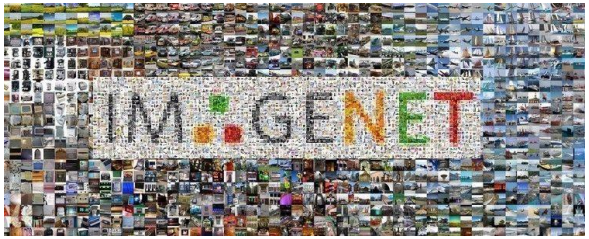


LeNet-5

A bit of history:

ImageNet Classification with Deep Convolutional Neural Networks

[Krizhevsky, Sutskever, Hinton, 2012]



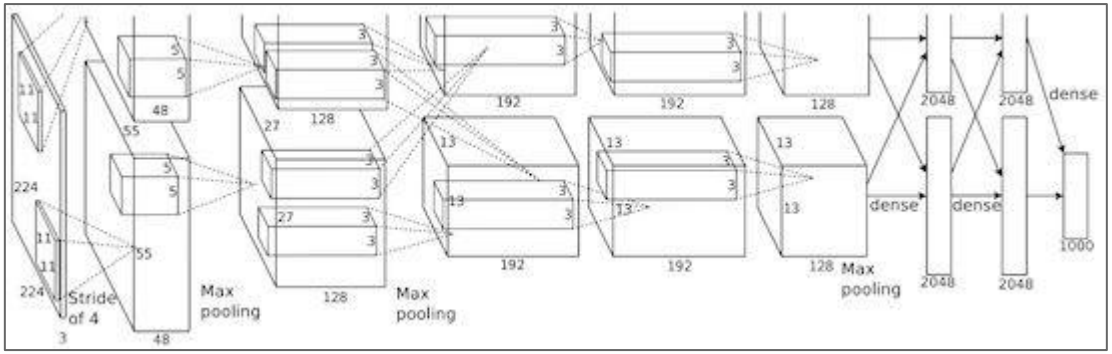


Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

"AlexNet"

Fast-forward to today: ConvNets are everywhere

Classification

Retrieval

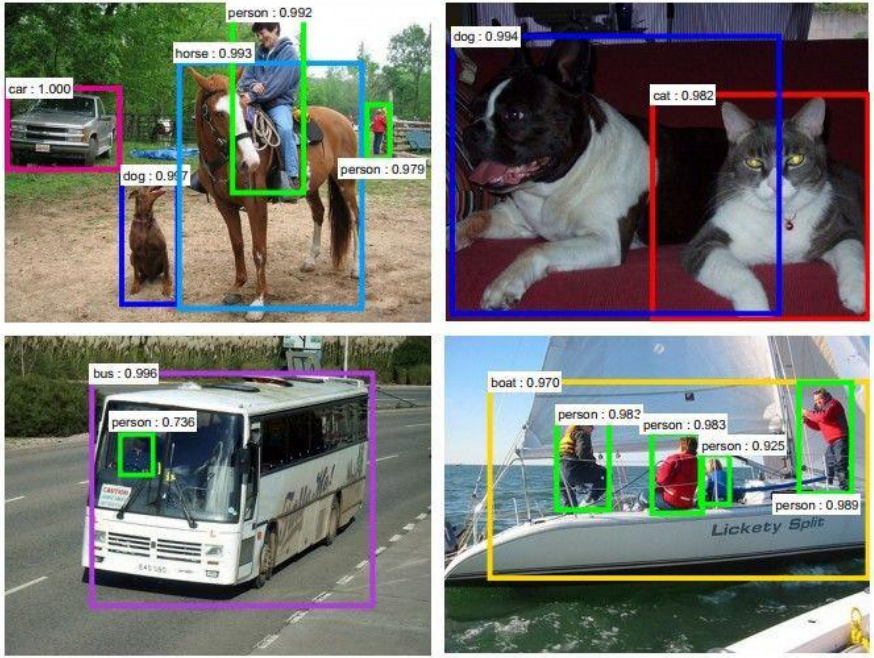




Figures copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

Fast-forward to today: ConvNets are everywhere

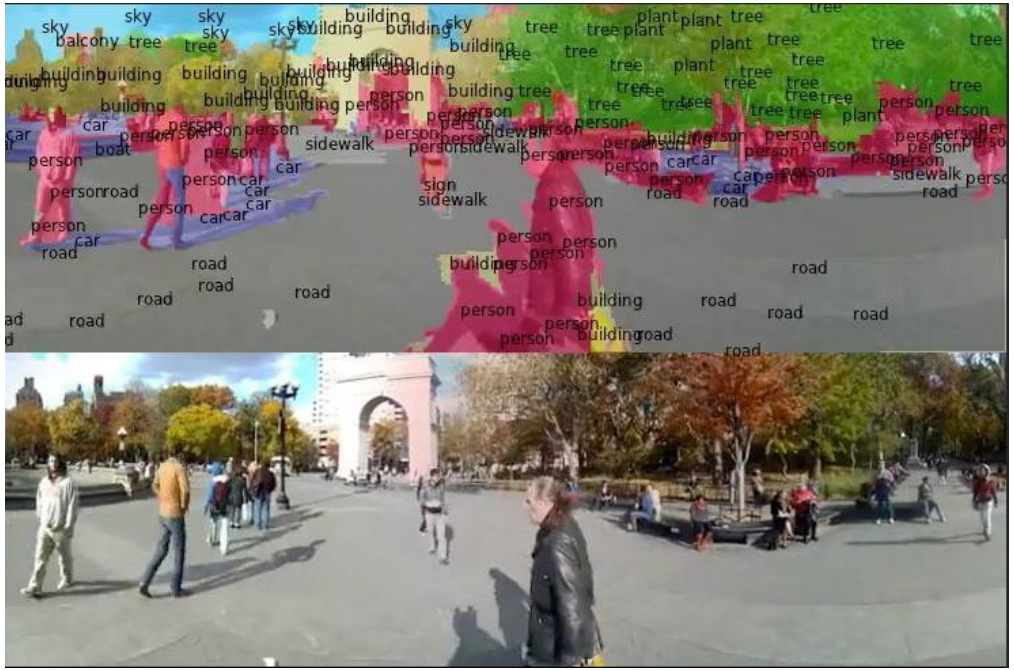
Detection



Figures copyright Shaoqing Ren, Kaiming He, Ross Girschick, Jian Sun, 2015. Reproduced with permission.

[Faster R-CNN: Ren, He, Girshick, Sun 2015]

Segmentation



Figures copyright Clement Farabet, 2012.
Reproduced with permission.

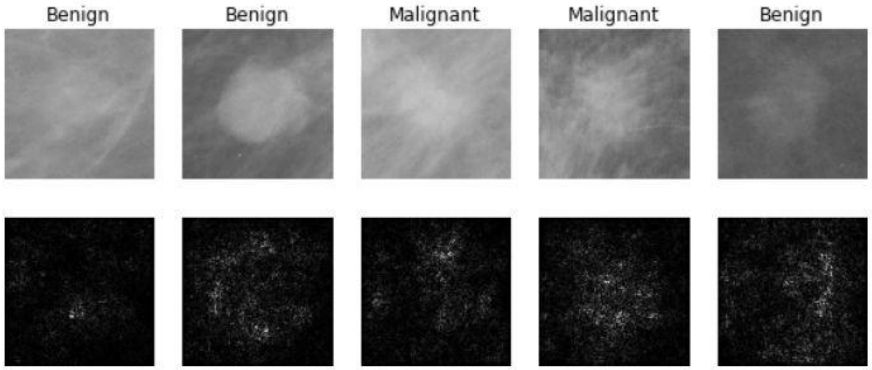
[Farabet et al., 2012]

Fast-forward to today: ConvNets are everywhere

self-driving cars



Fast-forward to today: ConvNets are everywhere



[Levy et al. 2016]

Figure copyright Levy et al. 2016.
Reproduced with permission.



[Dieleman et al. 2014]

From left to right: public domain by NASA, usage permitted by ESA/Hubble, public domain by NASA, and public domain.



[Sermanet et al. 2011]
[Ciresan et al.]

Photos by Lane McIntosh.
Copyright CS231n 2017.

Image Captioning

[Vinyals et al., 2015]
[Karpathy and Fei-Fei, 2015]

No errors



A white teddy bear sitting in the grass



A man riding a wave on top of a surfboard

Minor errors



A man in a baseball uniform throwing a ball

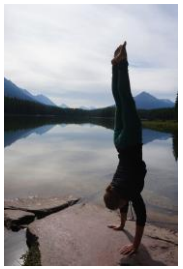


A cat sitting on a suitcase on the floor

Somewhat related



A woman is holding a cat in her hand



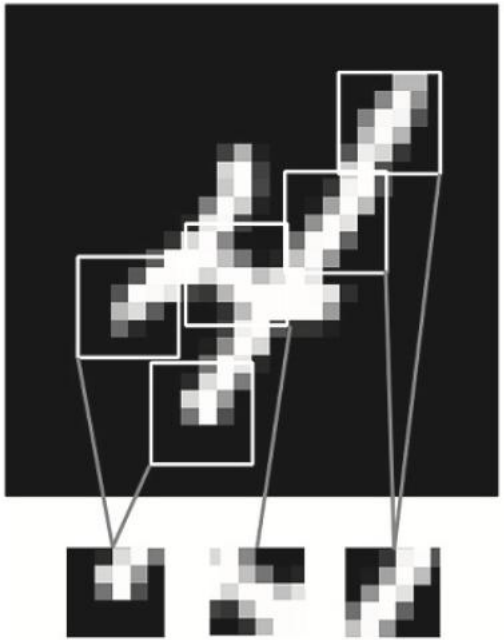
A woman standing on a beach holding a surfboard

Gatys et al, “Image Style Transfer using Convolutional Neural Networks”, CVPR 2016



Features in Images

- Digit classification can be performed by detecting certain discriminative features (image templates) occurring in their expected relative positions



Convolution in 1D

Continuous:

$$(x * w)(t) = \int_{-\infty}^{\infty} x(\tau)w(t-\tau)d\tau$$

Discrete:

$$(x * w)[n] = \sum_{k=-\infty}^{\infty} x[k]w[n-k]$$

- Flipping kernel w, sliding it over input x, computing elementwise multiplications, and summing the results
- Discrete convolution of x = [1; 2; 3; 4] with w = [5; 6; 7] to yield z = [5; 16; 34; 52; 45; 28]
- **Note: In deep learning frameworks or convolutional neural networks, the kernel is not flipped in practice — this operation is technically cross-correlation, not true convolution. However, since filters are learned, the distinction has no practical effect on the network's capability.**

Convolution in 2D

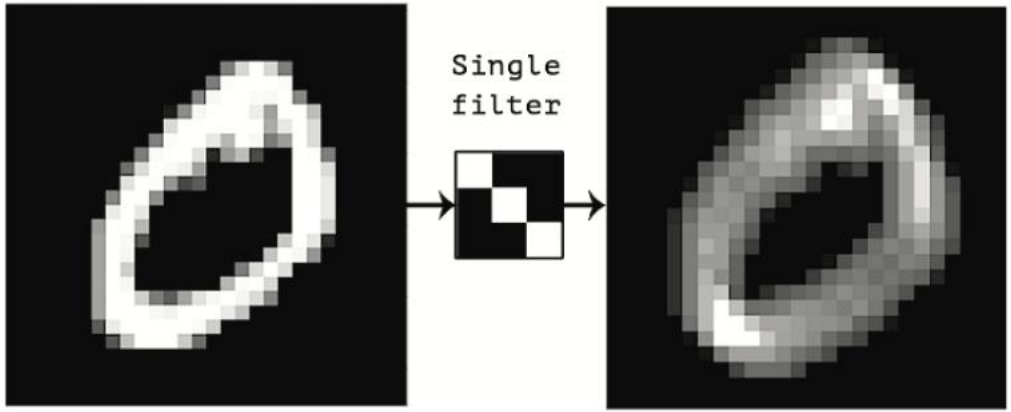
$$(x * w)[m, n] = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} x[i, j] w[m - i, n - j]$$



- Elementwise multiplication and sum of a filter and the signal (image)
- **(technically cross-correlation in practice — filter is not flipped in convolutional neural networks)**

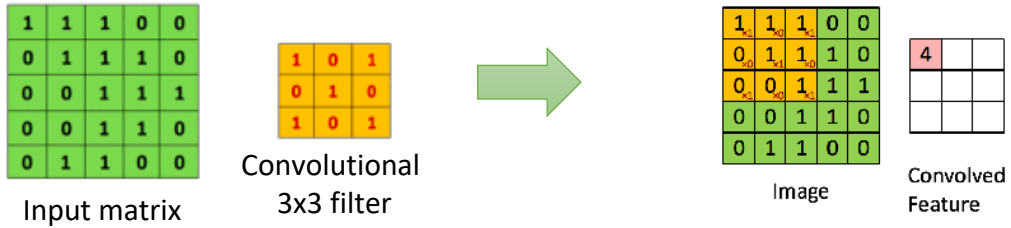
Convolution in 2D

- Convolving a 2D image (left) with a 3×3 filter (middle) produces a 2D response map (right)
- The bright spots in the response map indicate regions containing diagonal lines sloping downward and to the right



Convolutional Neural Networks (CNNs)

- ***Convolutional neural networks*** (CNNs) were primarily designed for image data
- CNNs use a convolutional operator for extracting data features
  - Allows parameter sharing
  - Efficient to train
  - Require fewer parameters than fully connected neural networks
- CNNs are robust to spatial translations of objects in images
- A convolutional filter slides (i.e., convolves) across the image
  - **In practice, this sliding operation is cross-correlation — the filter is applied as-is without flipping. Deep learning frameworks including PyTorch and TensorFlow implement it this way.**



Convolutional Neural Networks (CNNs)

- When the convolutional filters are scanned over the image, they capture useful features
  - Example: Edge detection using convolutional filters



Input Image

Filter (Laplacian kernel)



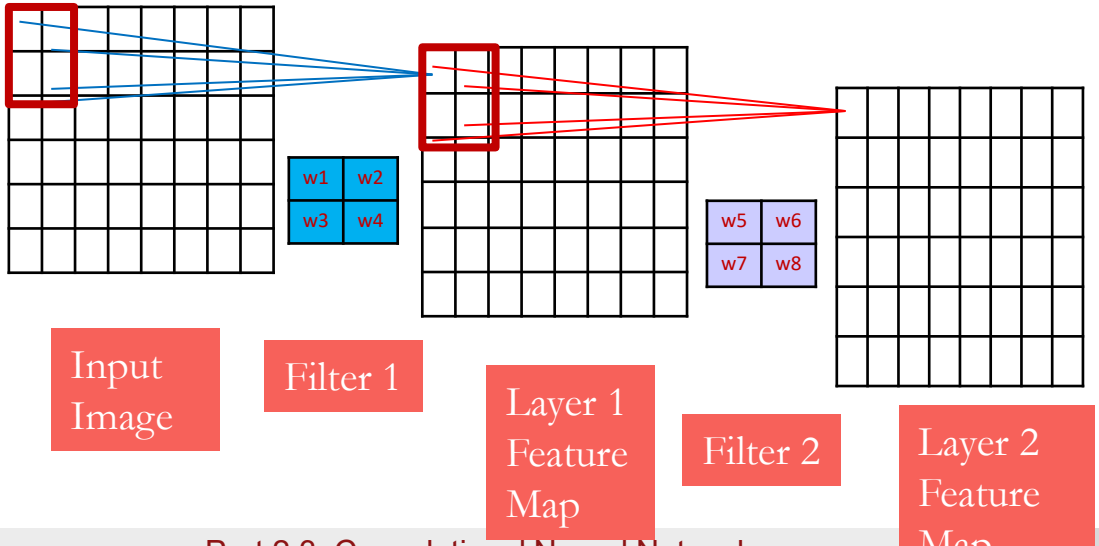
$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$



Convoluted Image

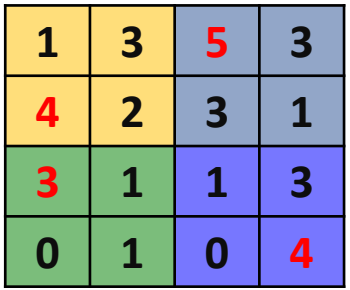
Convolutional Neural Networks (CNNs)

- In CNNs, neurons in a layer are only connected to a small spatial region of the previous layer, known as the local receptive field
  - The depth of each feature map corresponds to the number of convolutional filters used at each layer, e.g., RGB for color images, depth = 3



Pooling

- ***Max pooling***: Max pooling: Retains the maximum value within a defined spatial window
- ***Average pooling***: Computes the average value within a defined spatial window
- Pooling layers reduce the spatial size of the feature maps
  - Reduce the number of parameters, prevent overfitting



Input Matrix

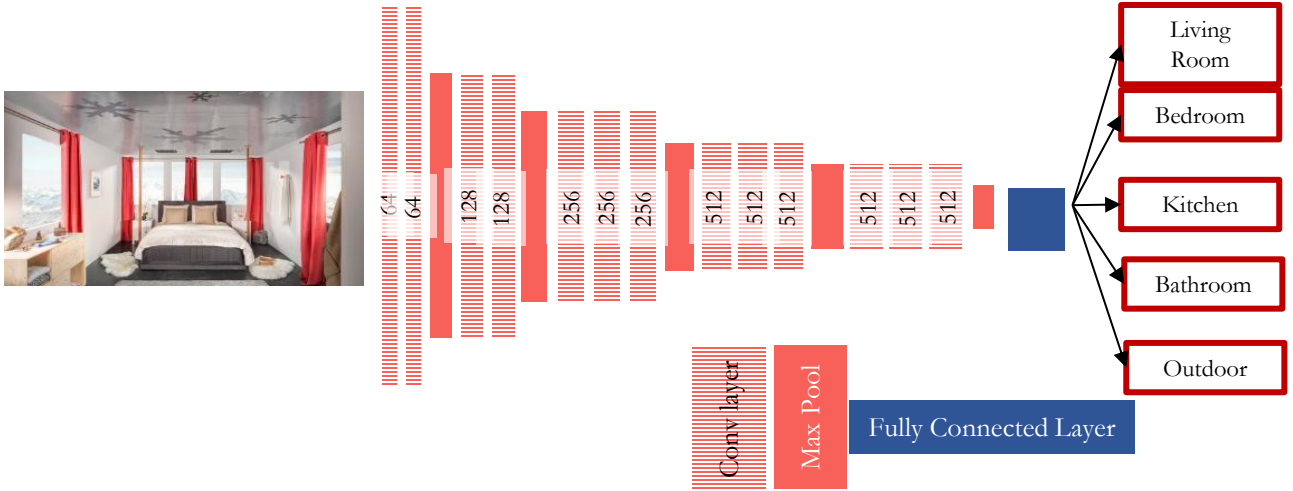


Output Matrix

MaxPool with a 2×2 filter with stride (step) of 2

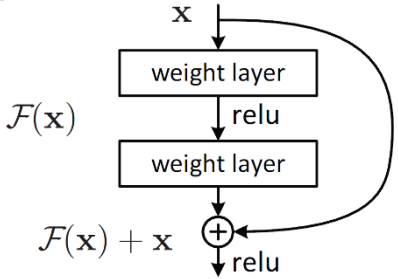
Feature Extraction Architecture

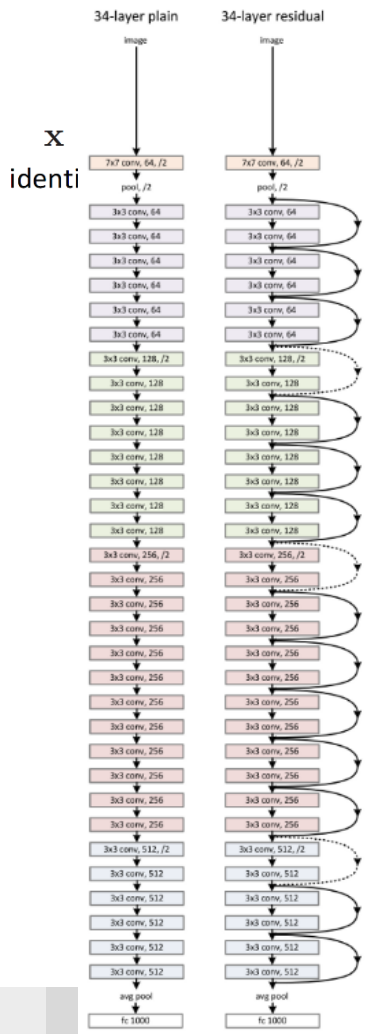
- After 2 convolutional layers, a max-pooling layer reduces the size of the feature maps (typically by 2)
- A fully convolutional and a softmax layers are added last to perform classification



Residual CNNs

- *Residual networks* (ResNets)
  - Introduce "identity" skip connections
    - Layer inputs are propagated and added to the layer output
    - Mitigate the problem of vanishing gradients during training
    - Allow training very deep NN (with over 1,000 layers)
  - Several ResNet variants exist: 18, 34, 50, 101, 152, and 200 layers
  - Are used as base models of other state-of-the-art NNs
    - Other similar models: ResNeXT, DenseNet

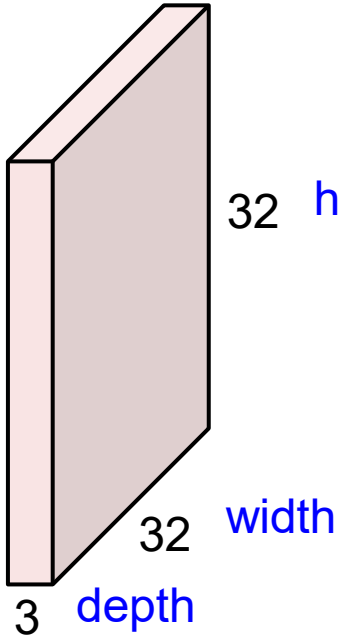




CNN Details

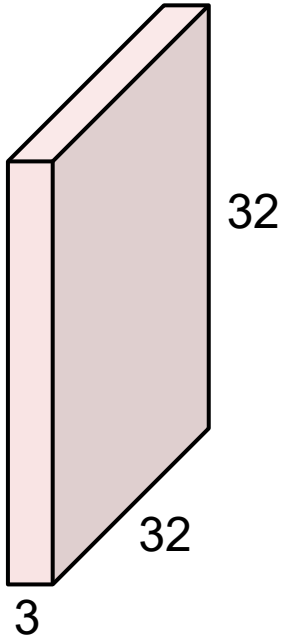
Convolution Layer

32x32x3 image -> preserve spatial structure

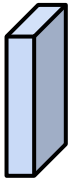


Convolution Layer

32x32x3 image



5x5x3 filter

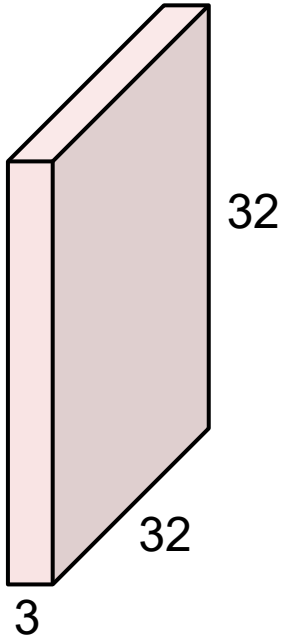


Convolve the filter with the image
i.e. “slide over the image spatially, computing dot products”

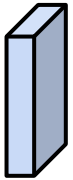
Convolution Layer

Filters always extend the full depth of the input volume

32x32x3 image

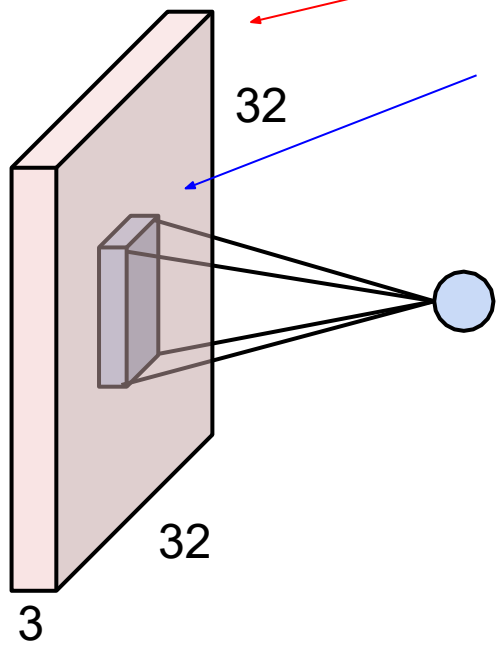


5x5x3 filter



Convolve the filter with the image i.e. “slide over the image spatially, computing dot products”

Convolution Layer



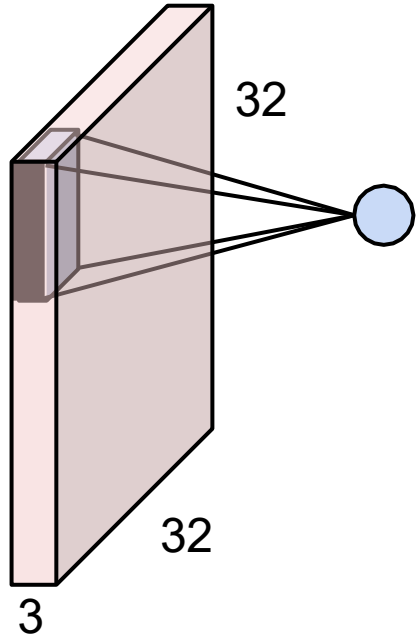
32x32x3 image

5x5x3 filter w

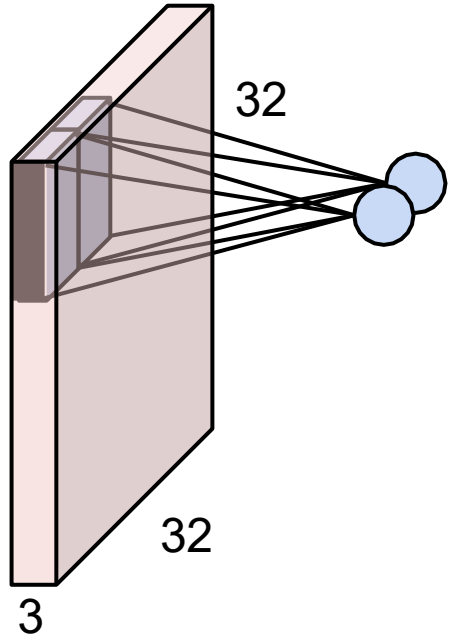
1 number:
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image (i.e. 5*5*3 = 75-dimensional dot product + bias)

$$w^T x + b$$

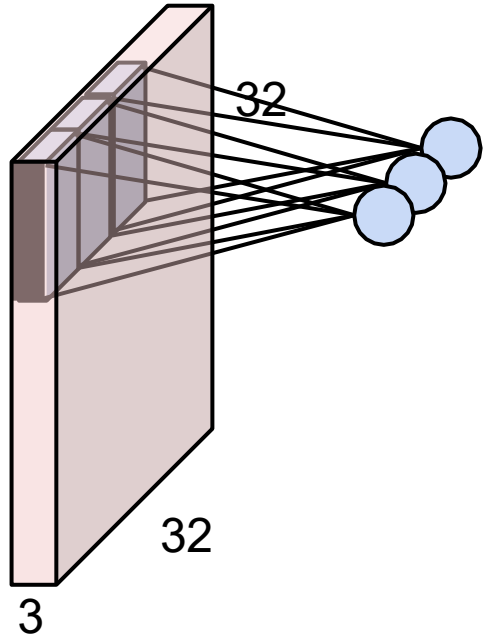
Convolution Layer



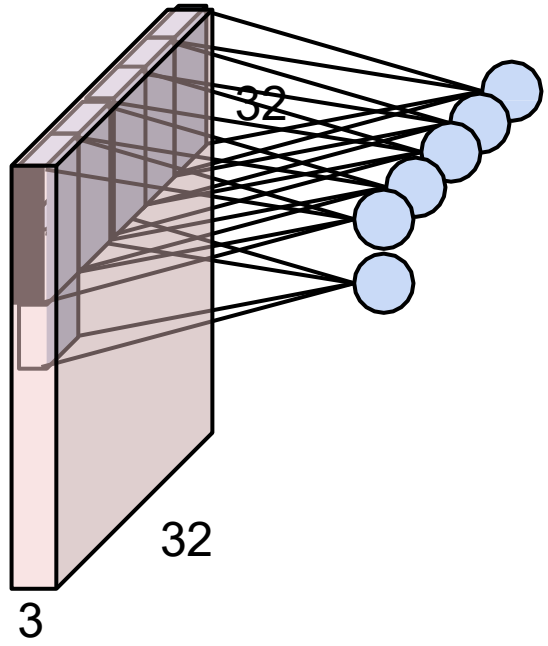
Convolution Layer



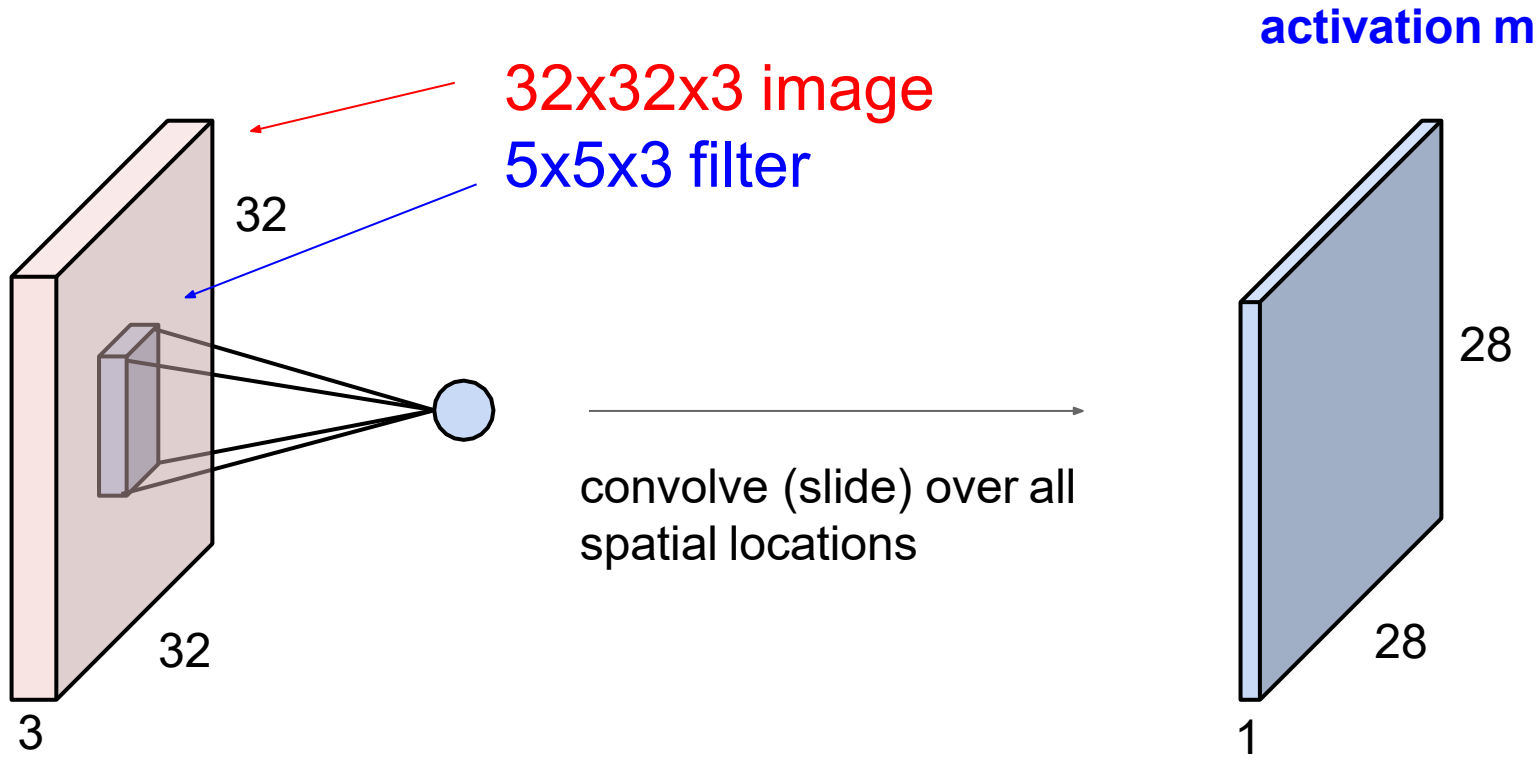
Convolution Layer



Convolution Layer

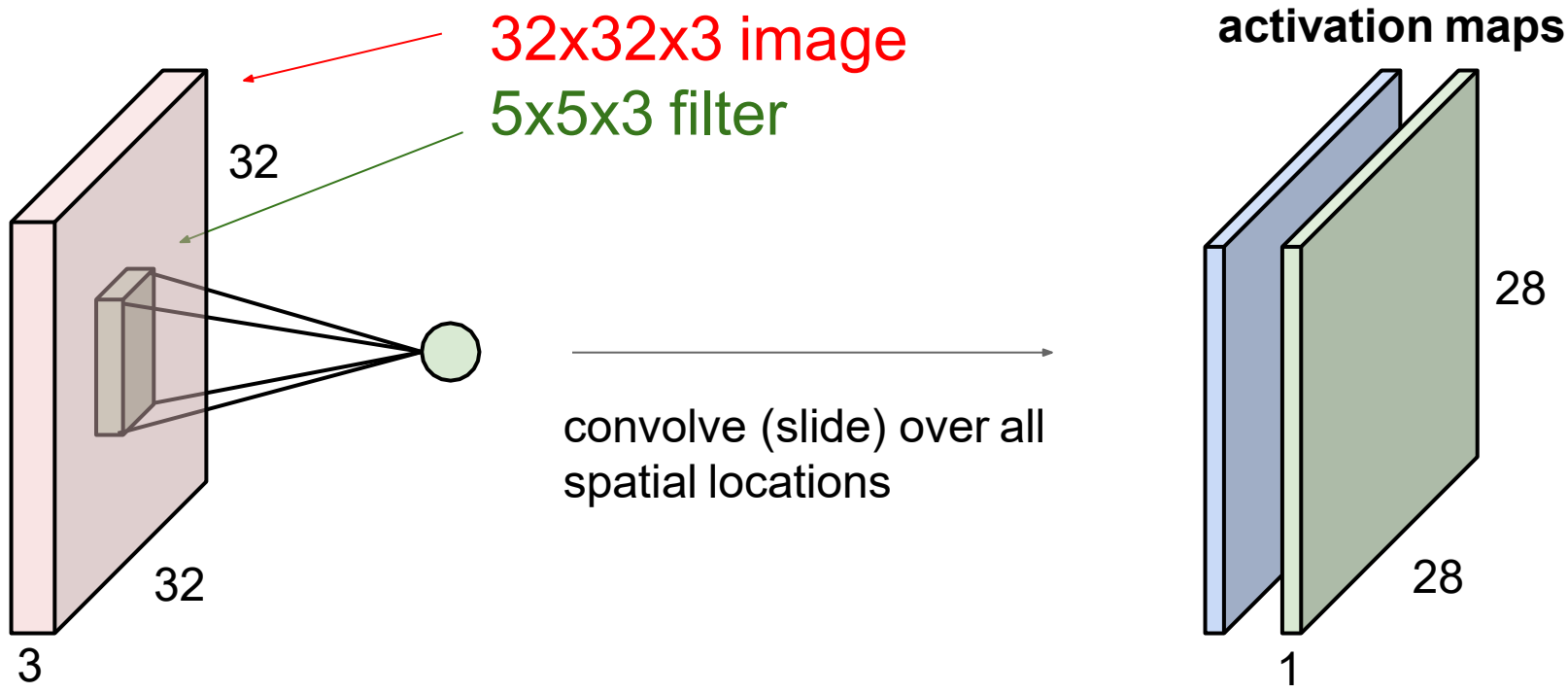


Convolution Layer

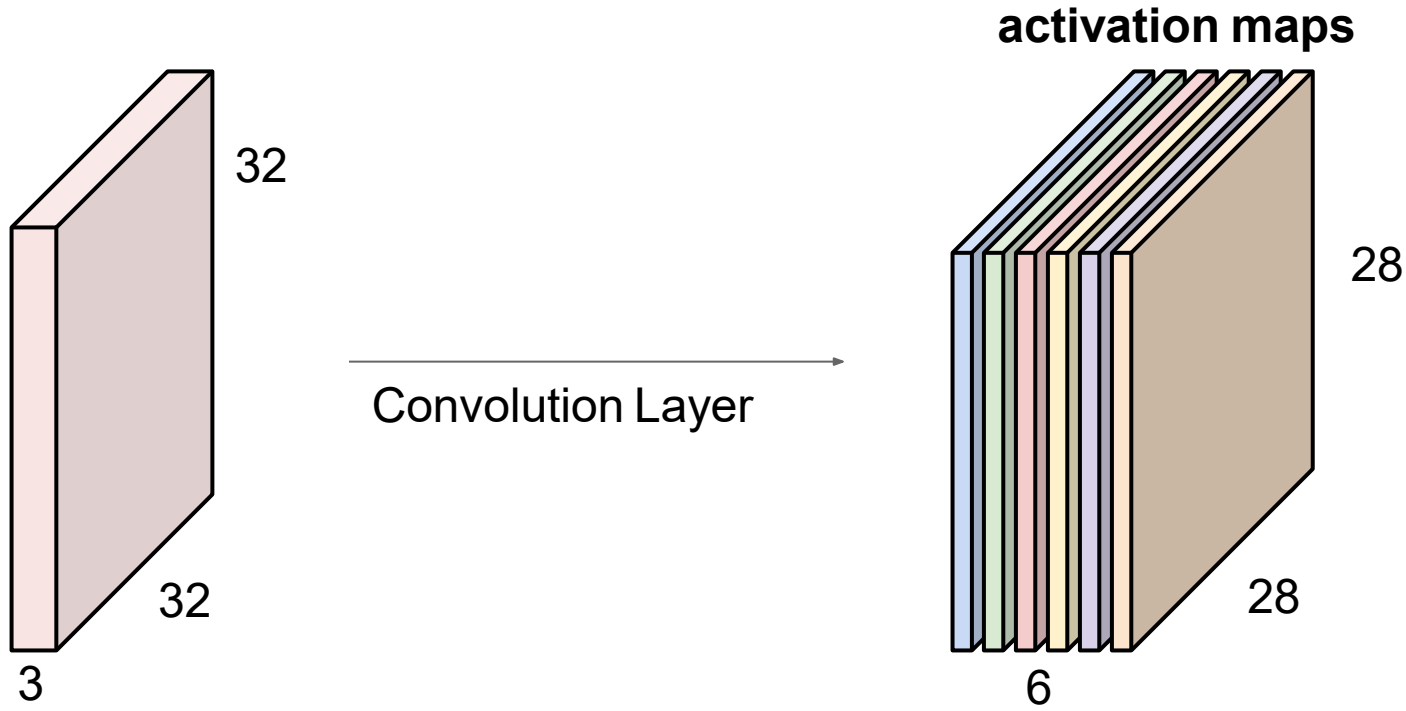


Convolution Layer

consider a second, green filter

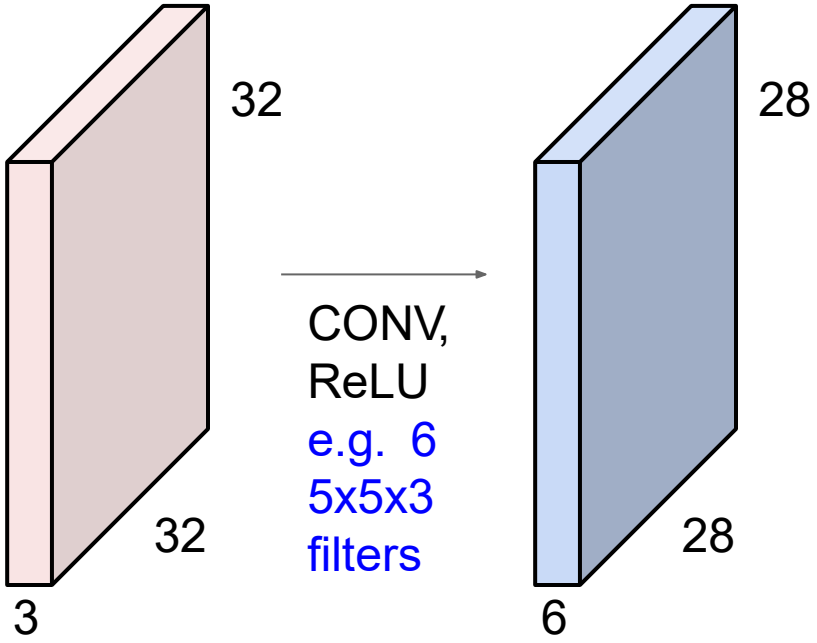


For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:

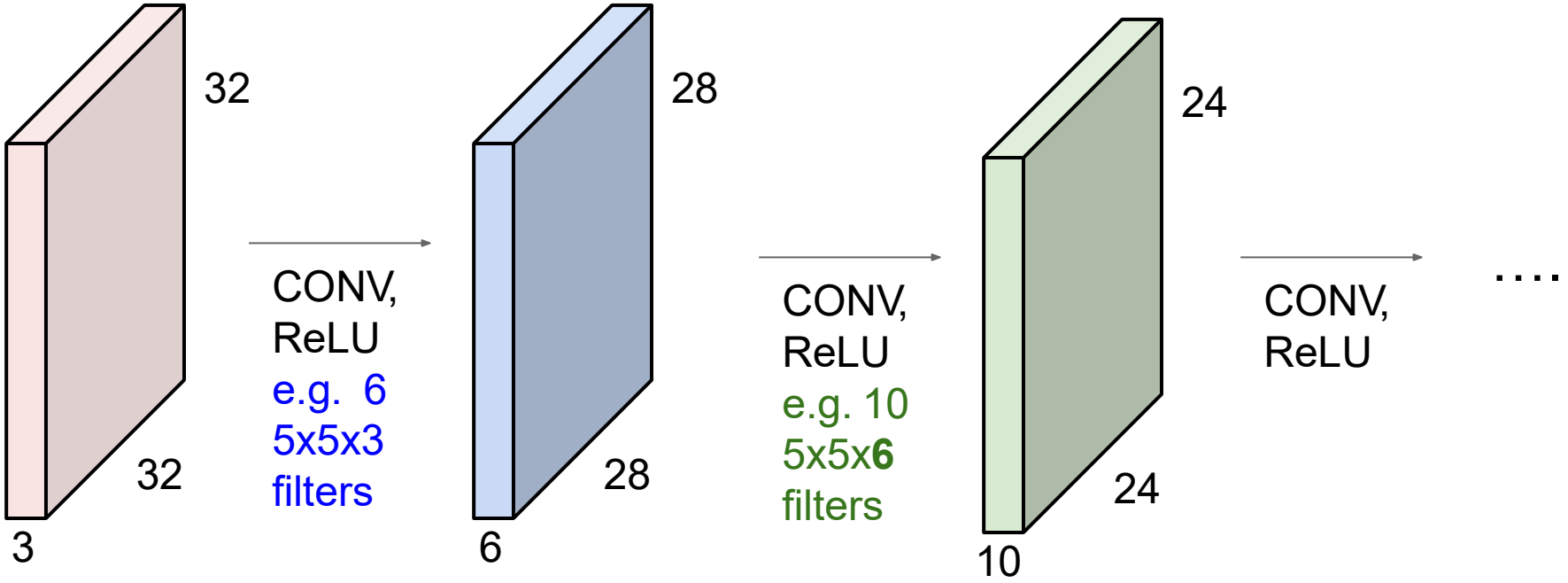


We stack these up to get a "new image" of size 28x28x6!

Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



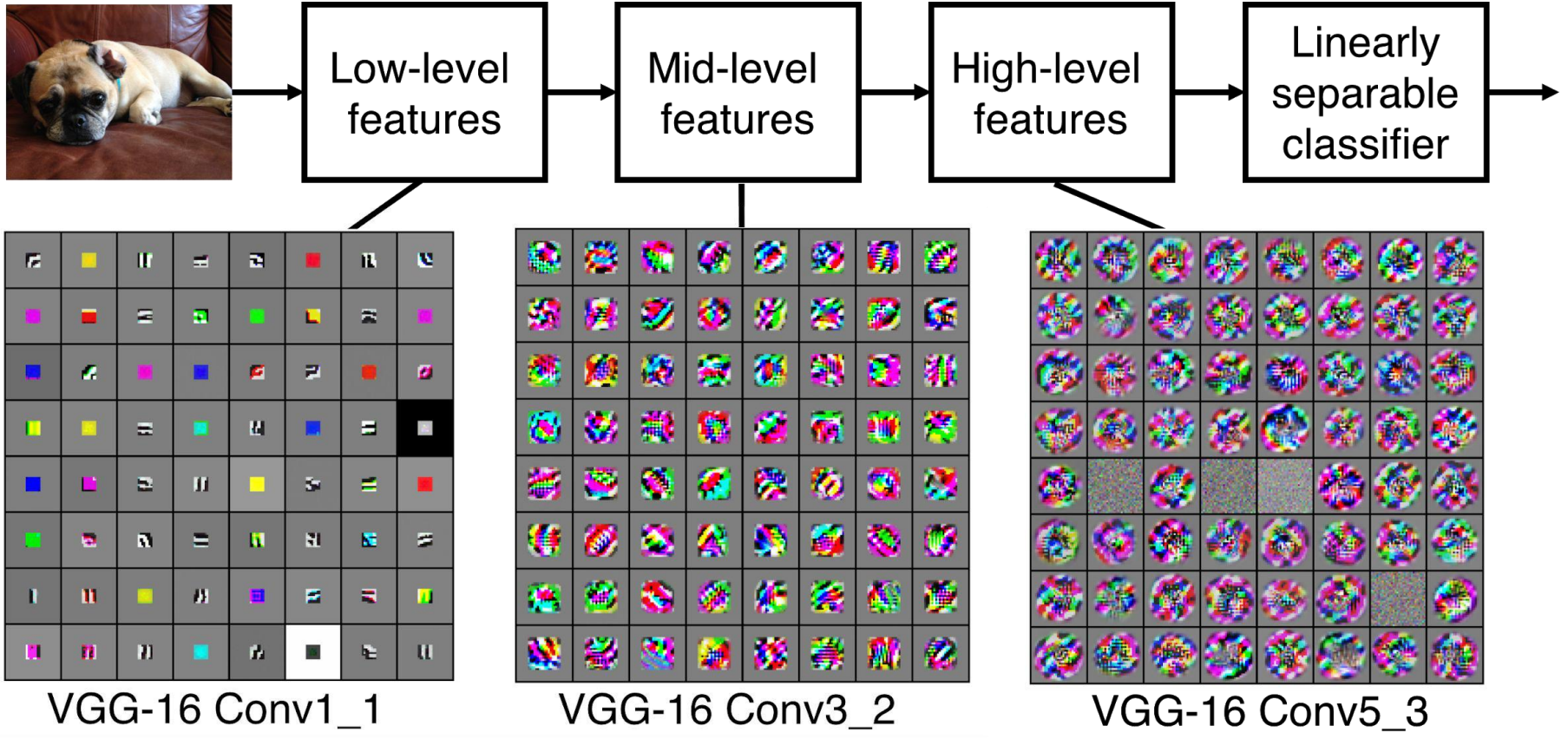
Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



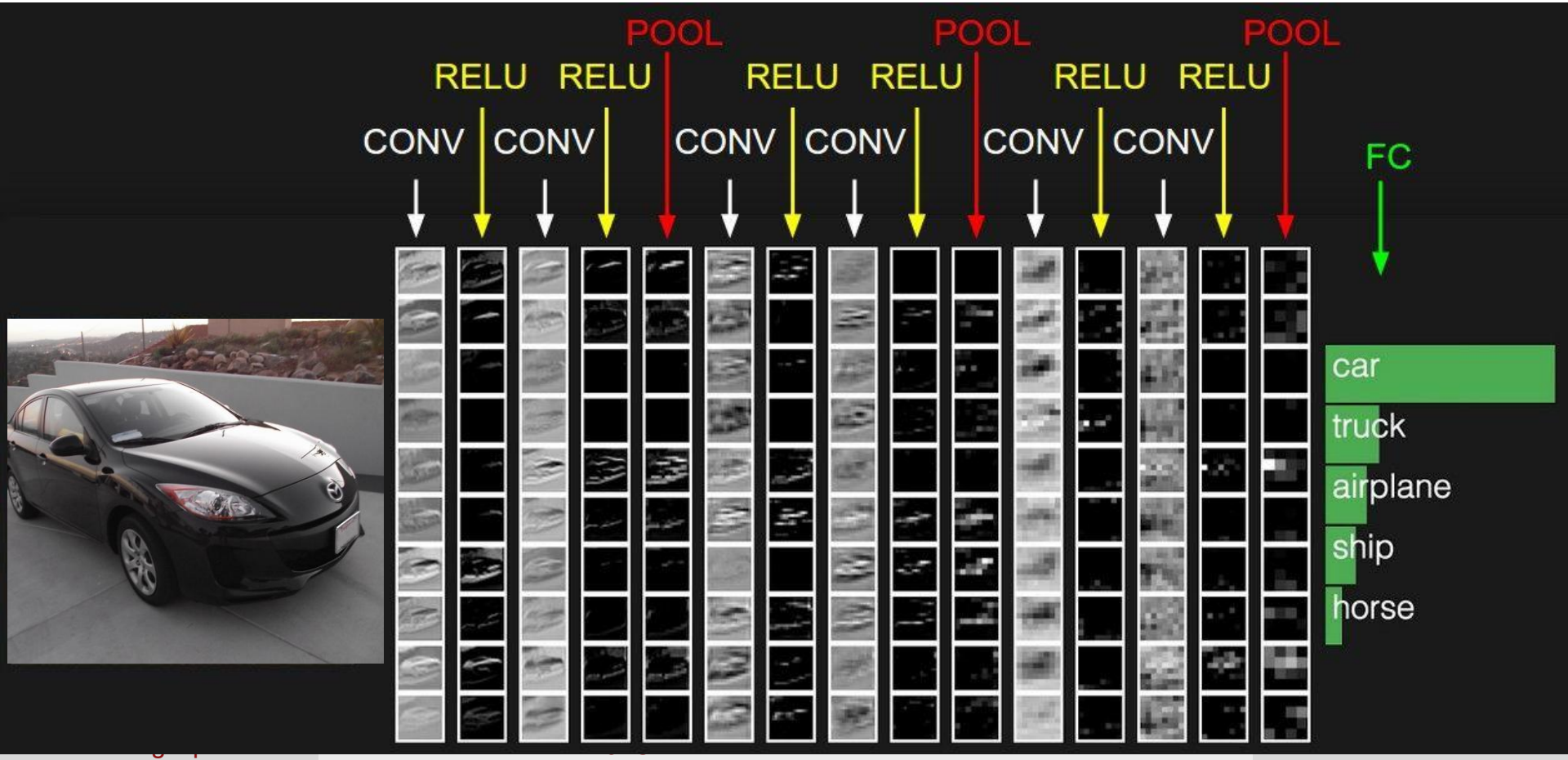
Preview

[Zeiler and Fergus 2013]

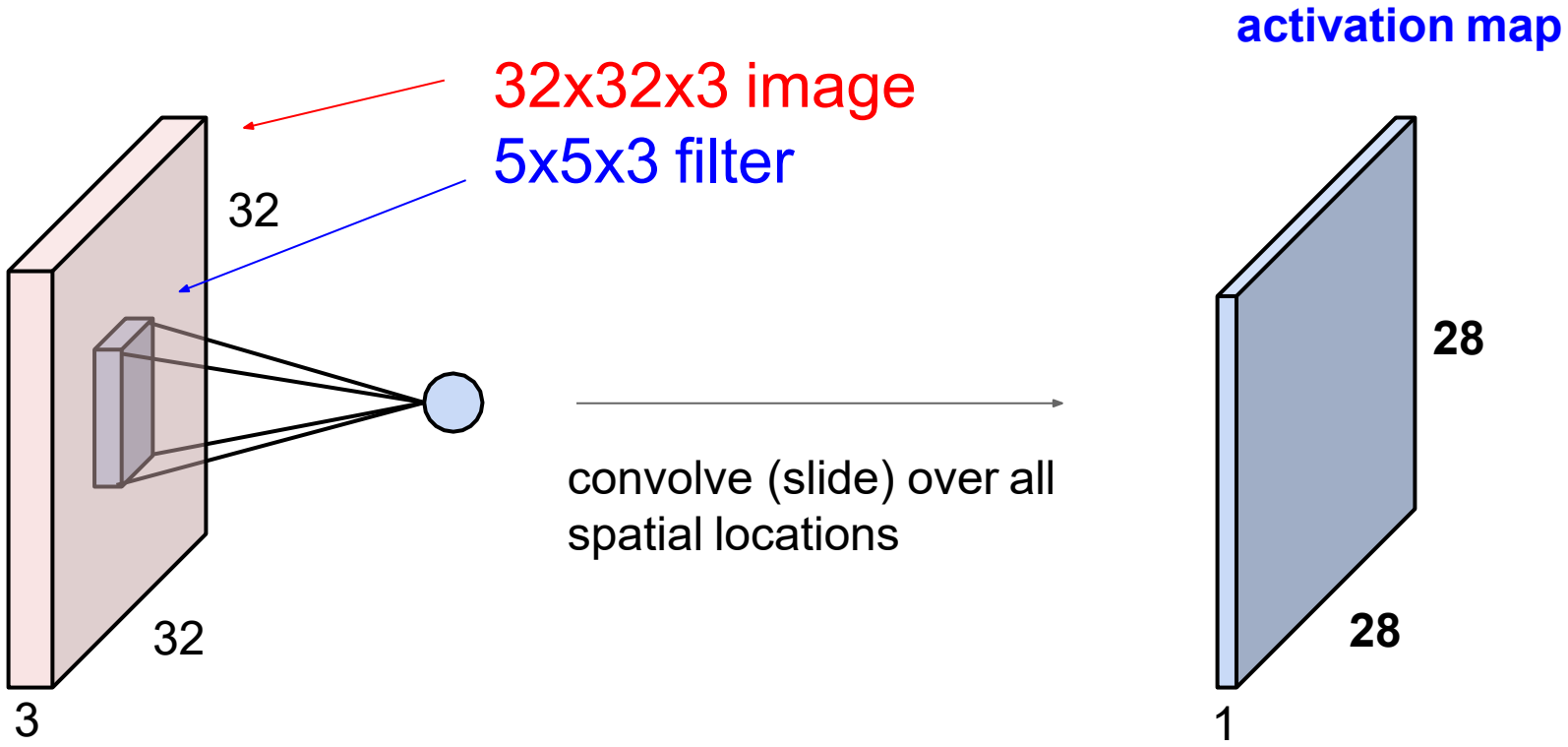
Visualization of VGG-16 by Lane McIntosh. VGG-16 architecture from [Simonyan and Zisserman 2014].



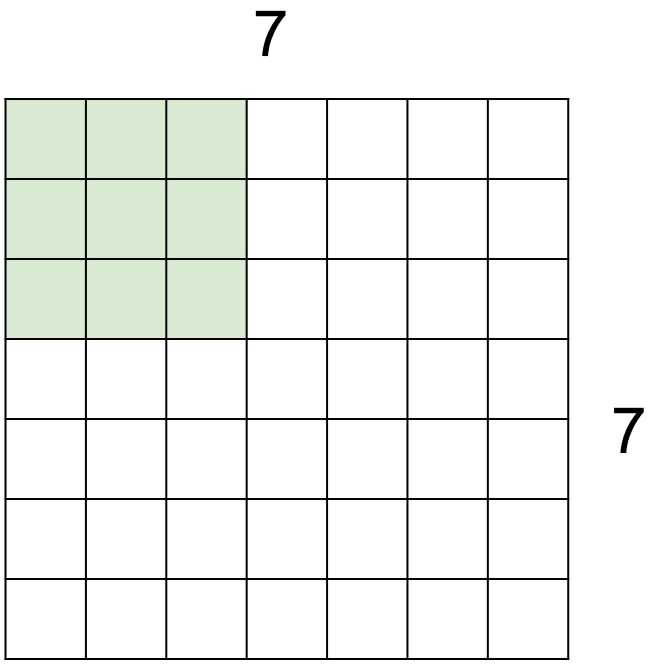
preview:


POOL
POOL
POOL
RELU
RELU
RELU
RELU
RELU
RELU
CONV
CONV
CONV
CONV
CONV
CONV
FC
car
truck
airplane
ship
horse

A closer look at spatial dimensions:

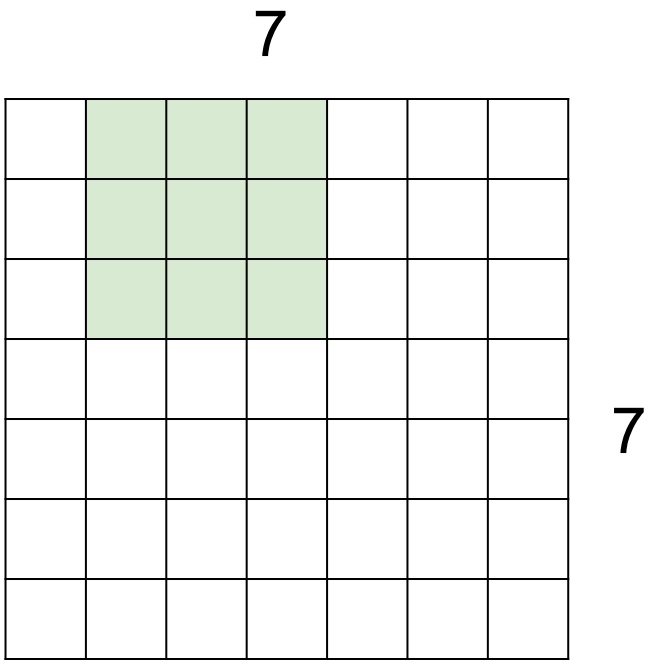


A closer look at spatial dimensions:



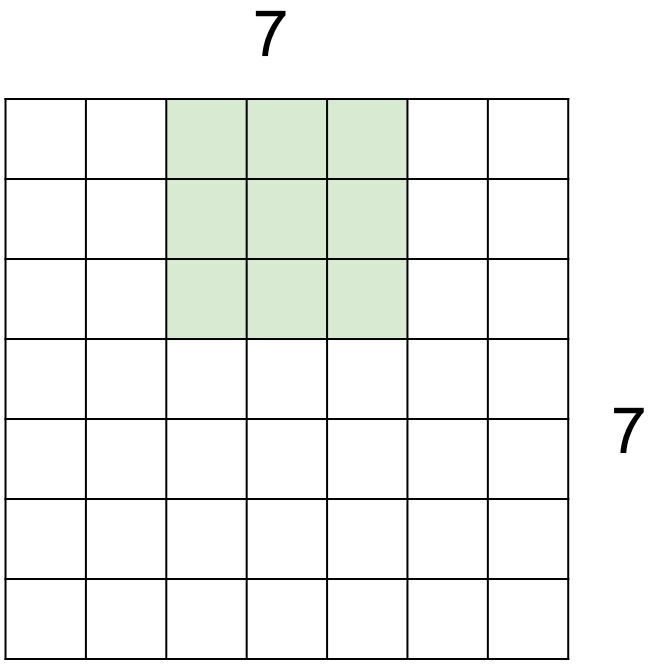
7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:



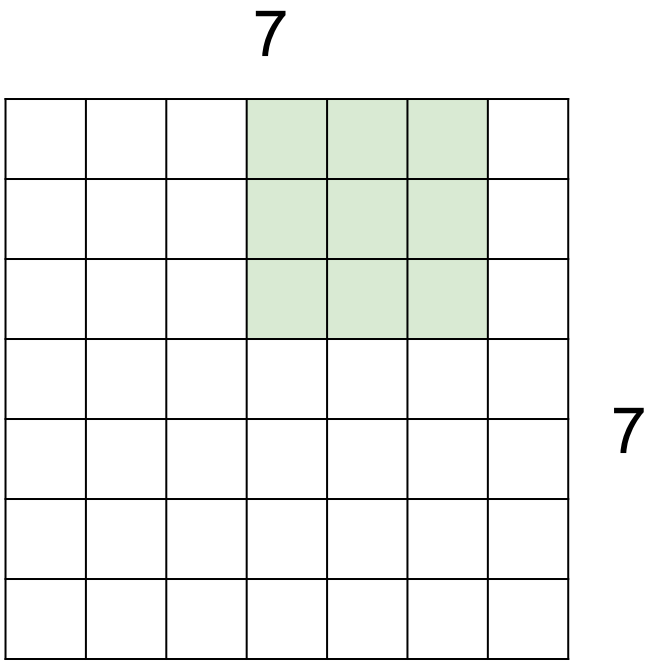
7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:



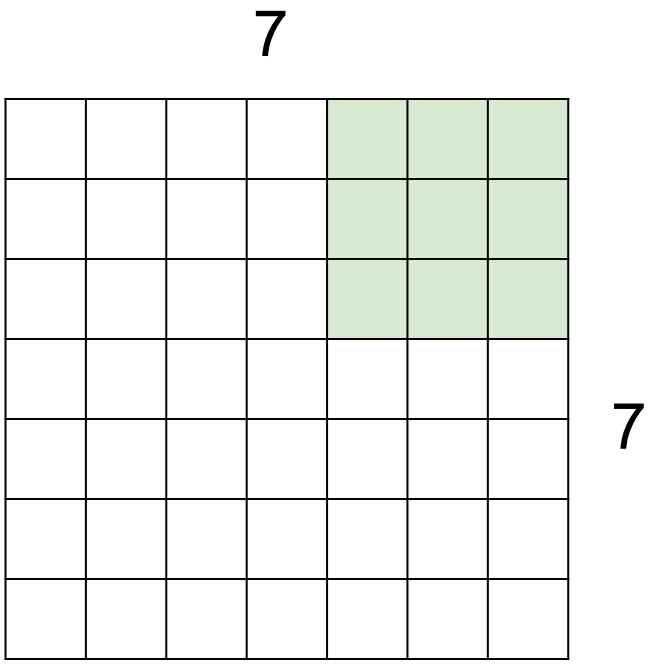
7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

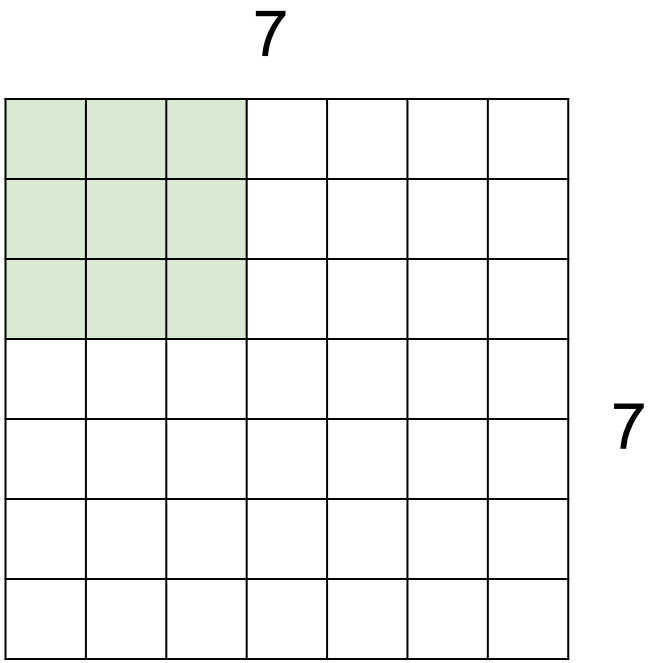
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

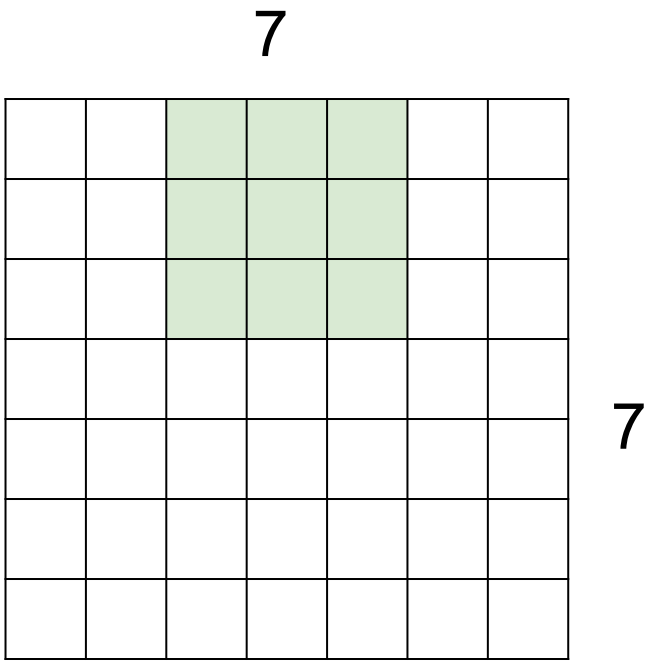
=> 5x5 output

A closer look at spatial dimensions:



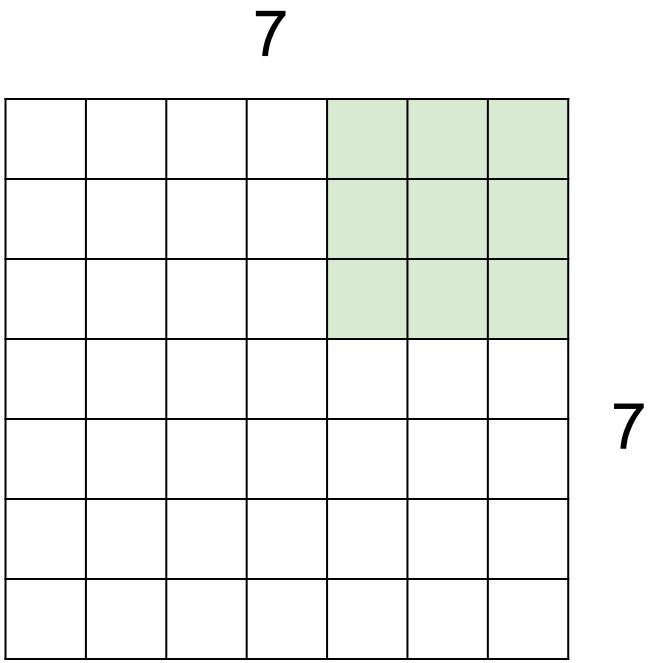
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions:



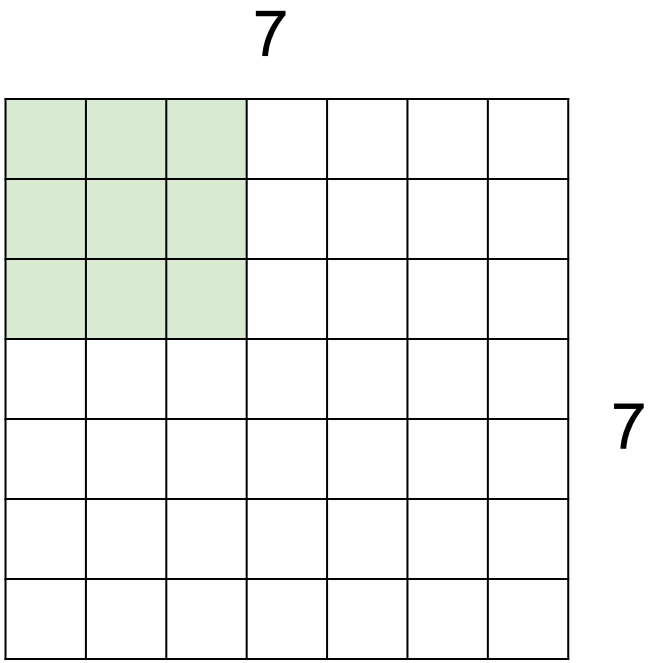
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions:



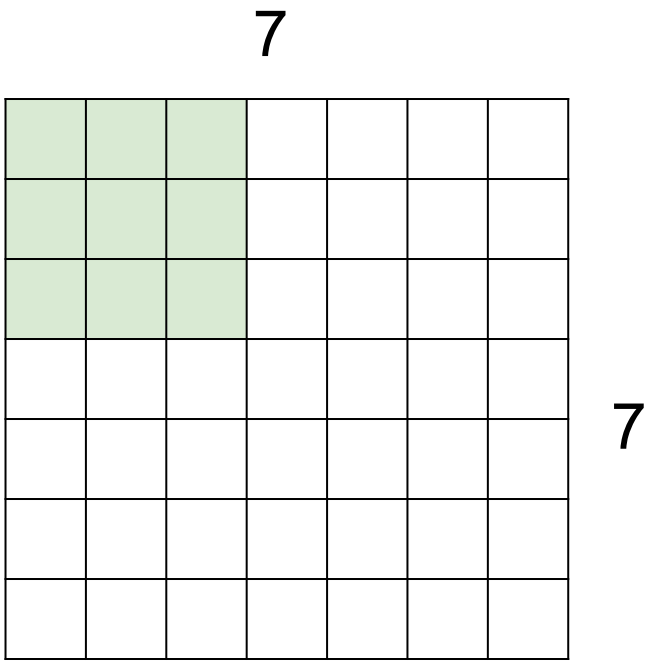
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!

A closer look at spatial dimensions:



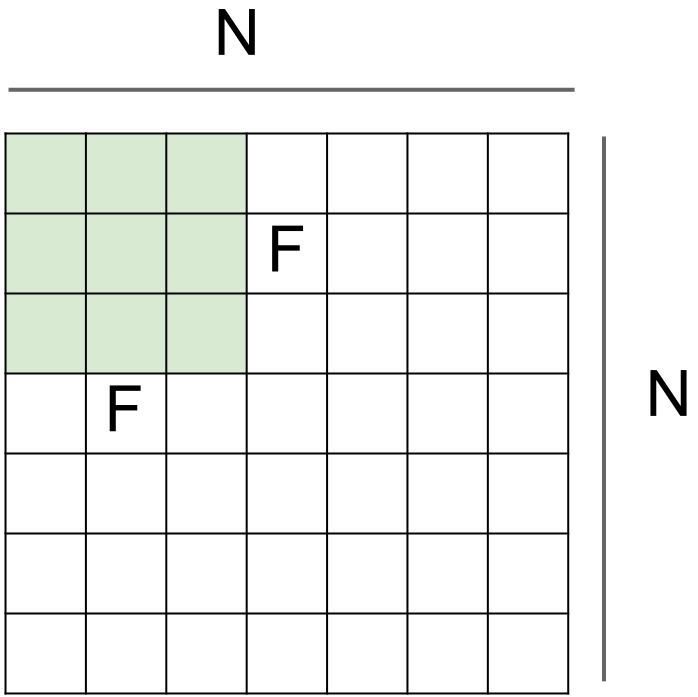
7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

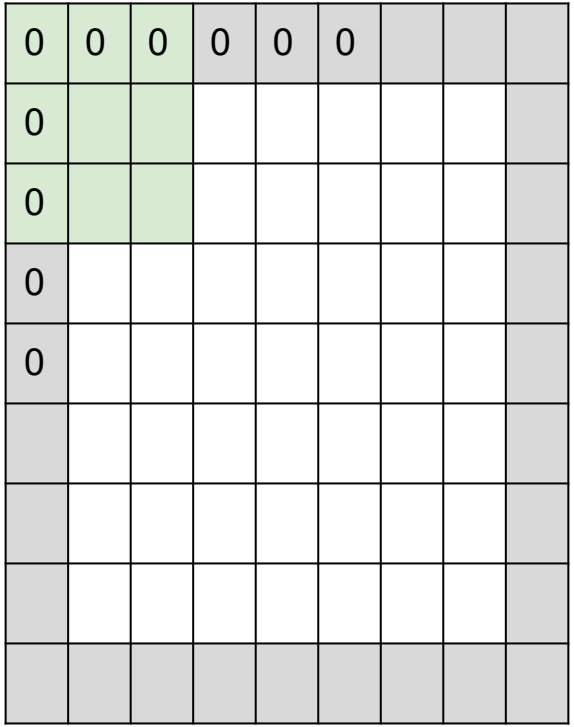
doesn't fit!
cannot apply 3x3 filter on
7x7 input with stride 3.



Output size:
(N - F) / stride + 1

e.g. N = 7, F = 3:
stride 1 => (7 - 3)/1 + 1 = 5
stride 2 => (7 - 3)/2 + 1 = 3
stride 3 => (7 - 3)/3 + 1 = 2.33 :\

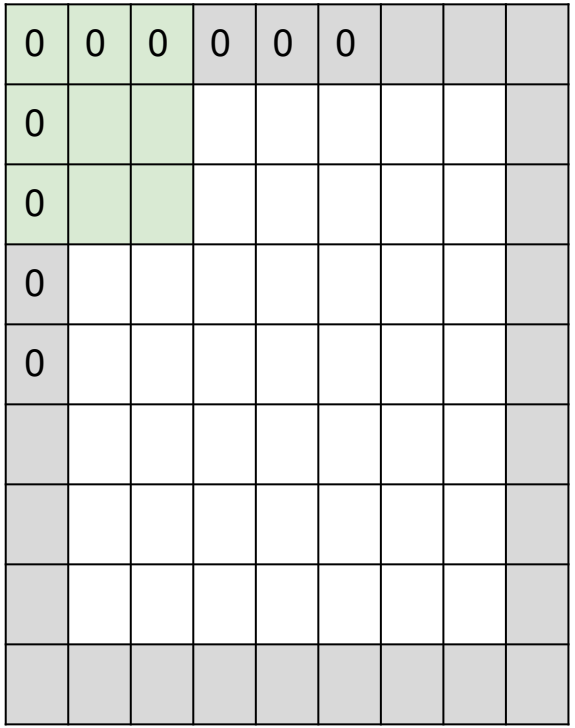
In practice: Common to zero pad the border



e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

(recall:)
(N - F) / stride + 1

In practice: Common to zero pad the border

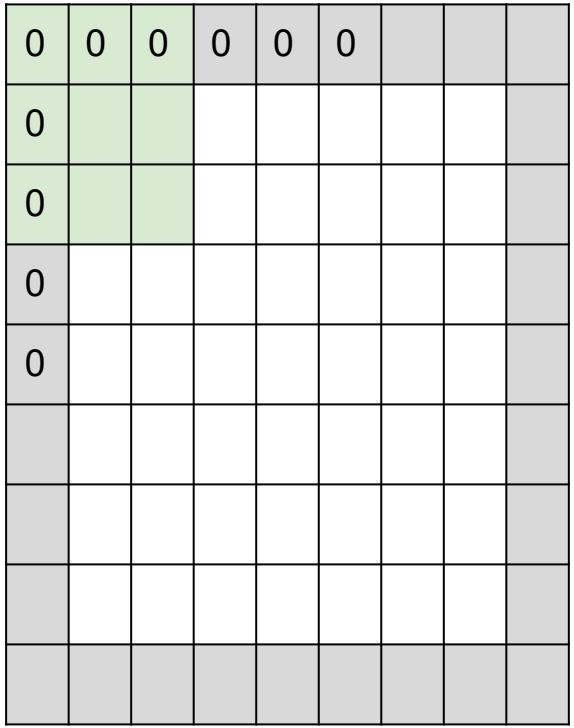


e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

7x7 output!

(recall:)
(N + 2P - F) / stride + 1

In practice: Common to zero pad the border

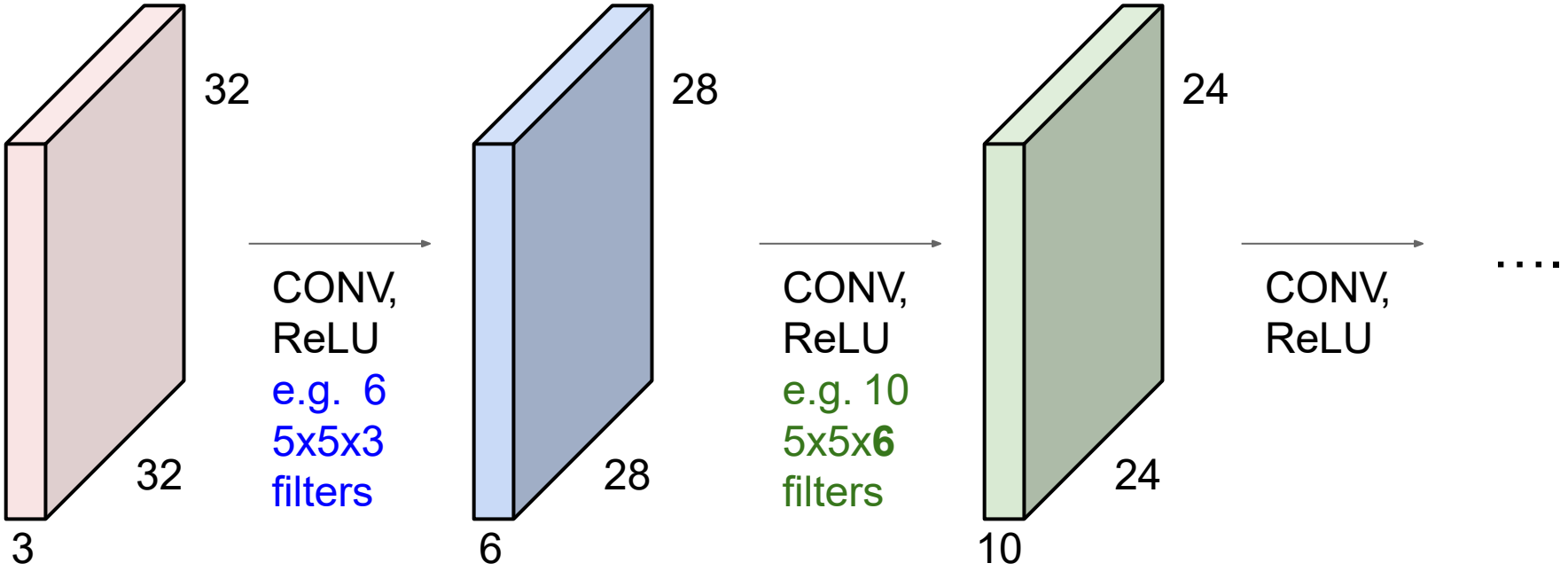


e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel of 0 around the border => what is the output size?
7x7 output!

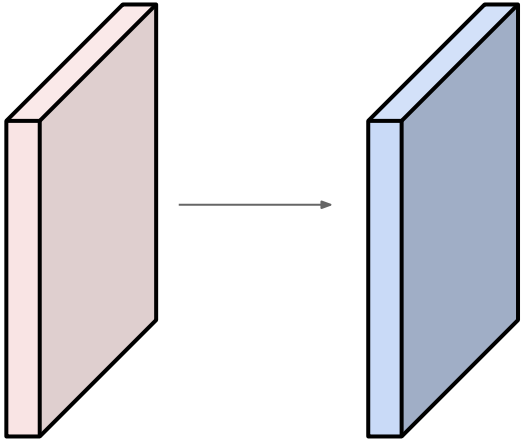
in general, common to see CONV layers with stride 1, filters of size FxF, and zero-padding with (F-1)/2 pixels. (will preserve size spatially)
e.g. F = 3 => zero pad with 1
F = 5 => zero pad with 2
F = 7 => zero pad with 3

Remember back to…

E.g. 32x32 input convolved repeatedly with 5x5 filters shrinks volumes spatially! (32 -> 28 -> 24 ...). Shrinking too fast is not good, doesn't work well.



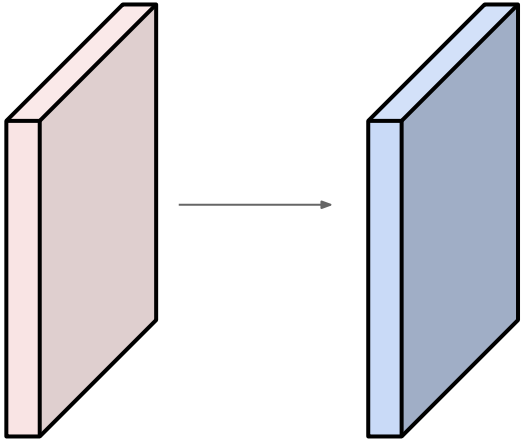
Example:



Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Output volume size: ?

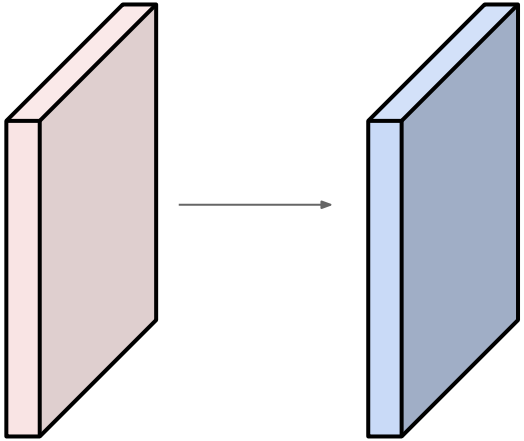
Example:



Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Output volume size:
(32+2*2-5)/1+1 = 32 spatially, so
32x32x10

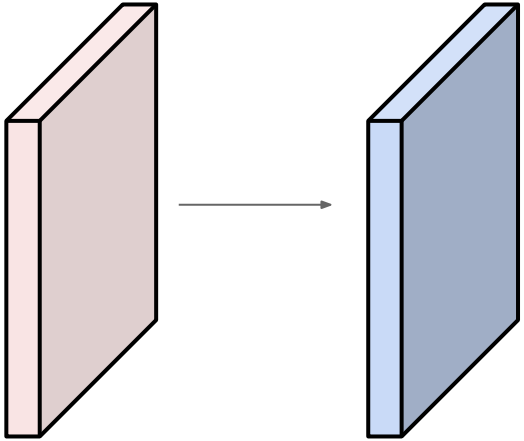
Example:



Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Number of parameters (weights + bias) in this layer?

Example:



Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Number of parameters in this layer?
each filter has 5*5*3 + 1 = 76 params (+1 for bias)
=> 76*10 = **760**

Convolution layer: summary

Let's assume input is W_1 x H_1 x C
Conv layer needs 4 hyperparameters:

- Number of filters **K**
- The filter size **F**
- The stride **S**
- The zero padding **P**

This will produce an output of W_2 x H_2 x K
where:

- $W_2 = (W_1 - F + 2P)/S + 1$
- $H_2 = (H_1 - F + 2P)/S + 1$

Number of parameters: $F^2CK + K$ biases

Convolution layer: summary

Common settings:

K = 32, 64, 128, 512
F = 3, S = 1, P = 1
F = 5, S = 1, P = 2
F = 5, S = 2, P = ? (whatever fits)
F = 1, S = 1, P = 0

Let's assume input is W_1 x H_1 x C
Conv layer needs 4 hyperparameters:

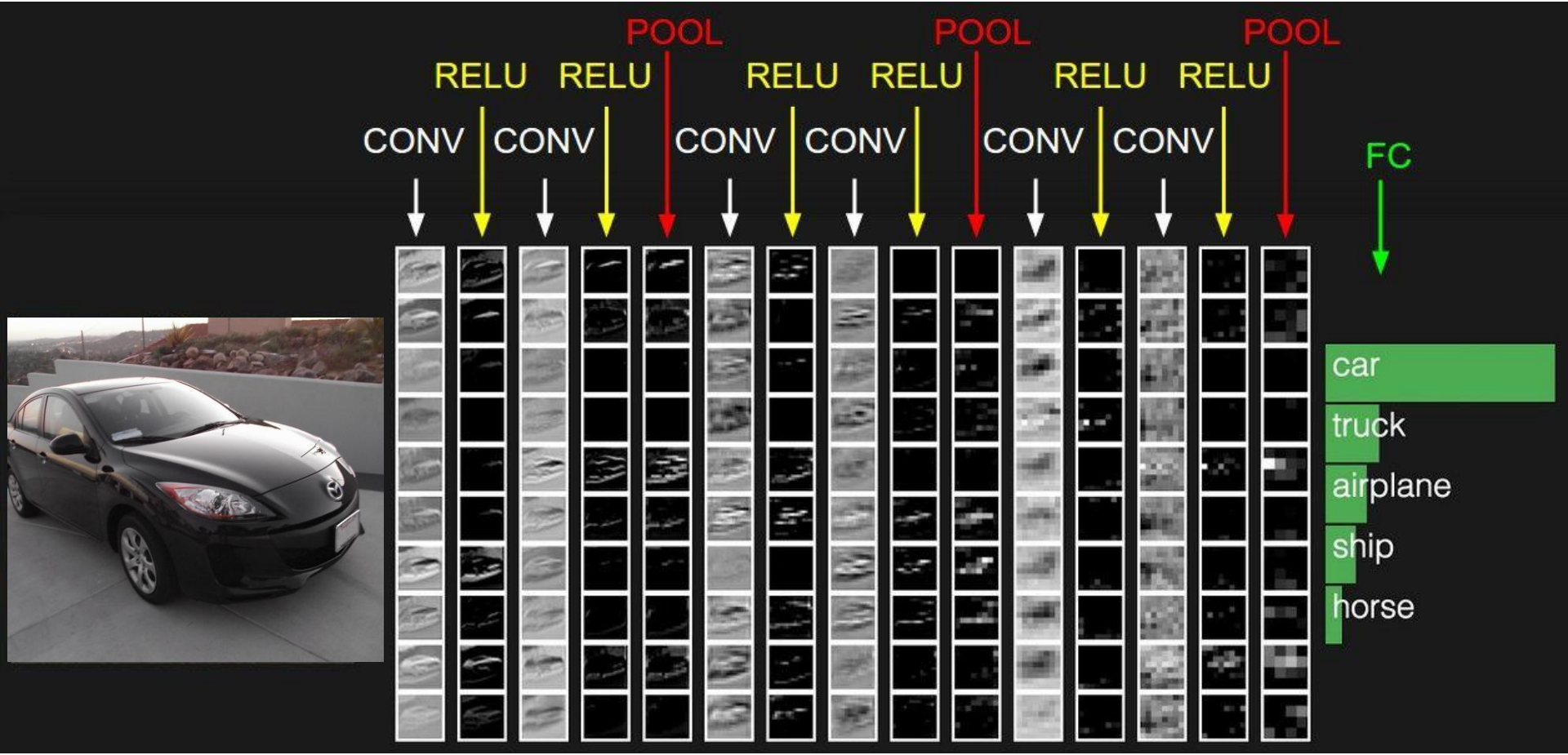
- Number of filters **K**
- The filter size **F**
- The stride **S**
- The zero padding **P**

This will produce an output of W_2 x H_2 x K
where:

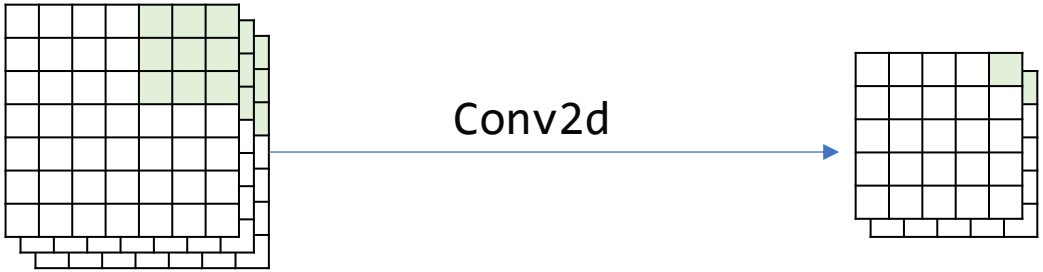
- $W_2 = (W_1 - F + 2P)/S + 1$
- $H_2 = (H_1 - F + 2P)/S + 1$

Number of parameters: F^2CK + K biases

two more layers to go: POOL/FC



PyTorch Convolution



Denotes where the kernel perfectly overlaps all green cells

```
import torch

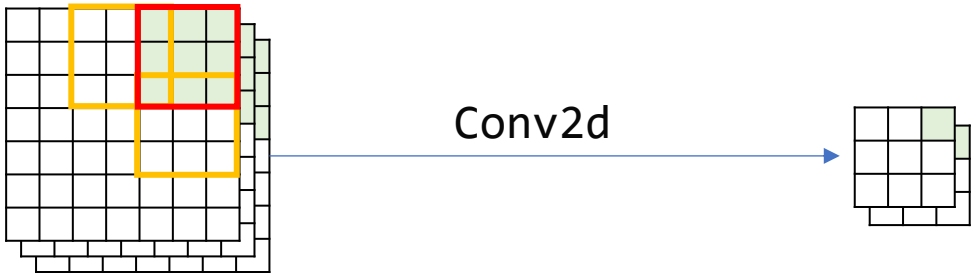
# Define input tensor
channels, height, width = 3, 7, 7
x = torch.randn(channels, height, width)

# Define convolutional layer
conv1 = torch.nn.Conv2d(in_channels=3, out_channels=2, kernel_size=3,
stride=1, padding=0)

# Print output shape
print(conv1(x.unsqueeze(0)).shape)  # Adds batch dimension
```

Despite being named Conv2d, PyTorch implements cross-correlation — the kernel is not flipped before computing dot products. This is standard across all major deep learning frameworks.

PyTorch Convolution



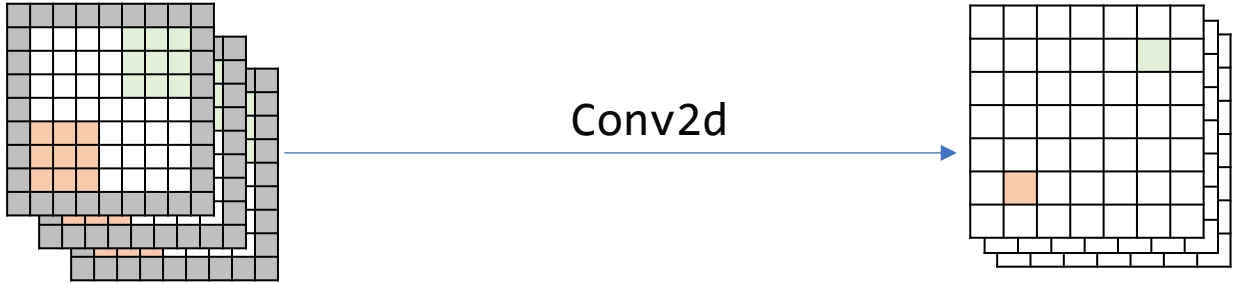

```
>>> import torch

>>> channels = 3
>>> height, width = 7, 7
>>> x = torch.randn([channels, height, width])
>>> conv1 = torch.nn.Conv2d(
>>>    in_channels=3, out_channels=2,
>>>    kernel_size=3, stride=2, padding=0
>>> )
>>> print(conv1(x).shape)

[2, 3, 3]
```

Denotes where the kernel perfectly overlaps all green cells

PyTorch Convolution



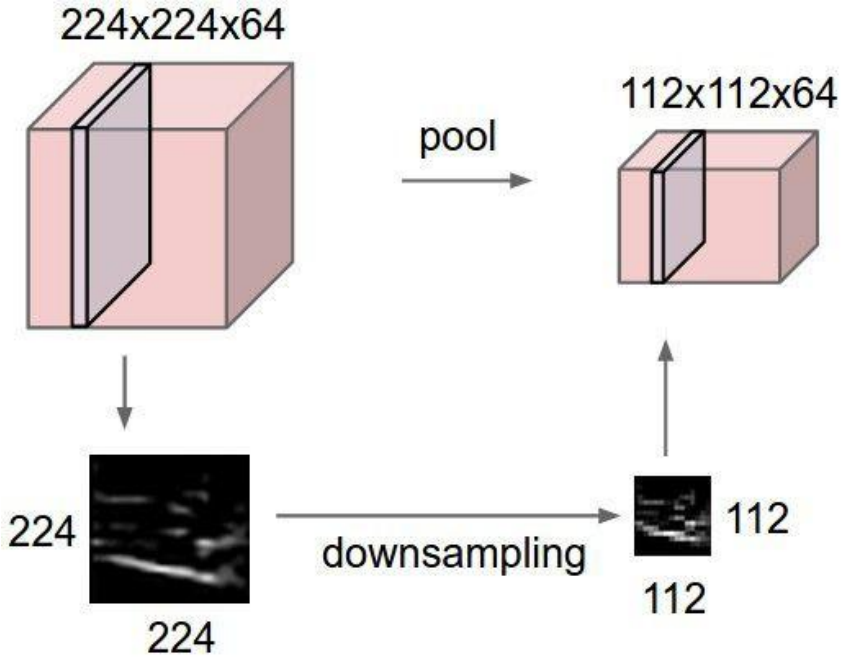
Denotes where the kernel perfectly overlaps all green /orange cells

```
>>> import torch

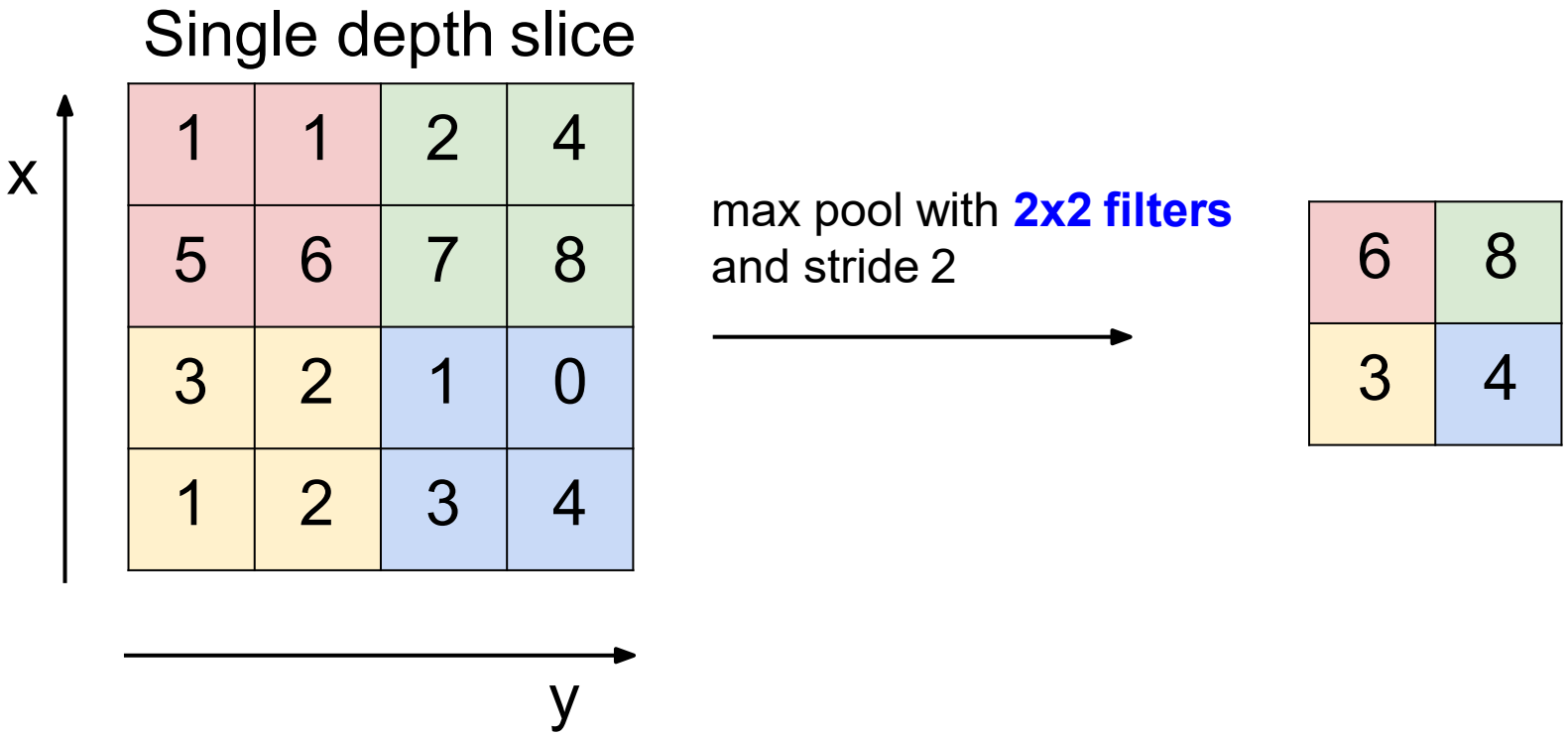
>>> channels = 3
>>> height, width = 7, 7
>>> x = torch.randn([channels, height, width])
>>> conv1 = torch.nn.Conv2d(
>>>     in_channels=3, out_channels=2,
>>>     kernel_size=3, stride=1, padding=1
>>> )
>>> print(conv1(x).shape)
[2, 7, 7]
```

Pooling layer

- makes the representations smaller and more manageable
- operates over each activation map independently:



MAX POOLING


Single depth slice
x
y
1 1 2 4
5 6 7 8
3 2 1 0
1 2 3 4
max pool with 2x2 filters
and stride 2
6 8
3 4

Pooling layer: summary

Let's assume input is W_1 x H_1 x C
Conv layer needs 2 hyperparameters:

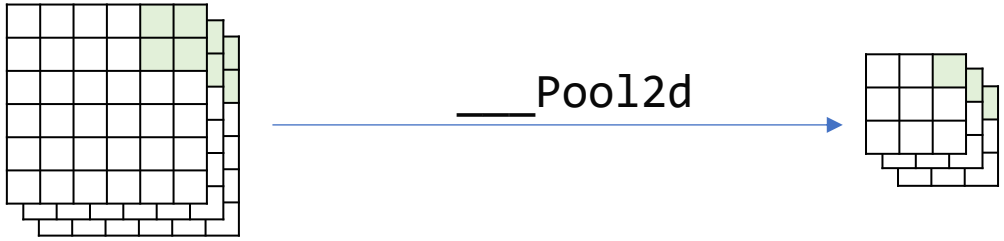
- The spatial extent (filter size) **F**
- The stride **S**

This will produce an output of W_2 x H_2 x C where:

- $W_2 = (W_1 - F)/S + 1$
- $H_2 = (H_1 - F)/S + 1$

Number of parameters: 0

PyTorch Pooling



Denotes where the kernel perfectly overlaps all green cells

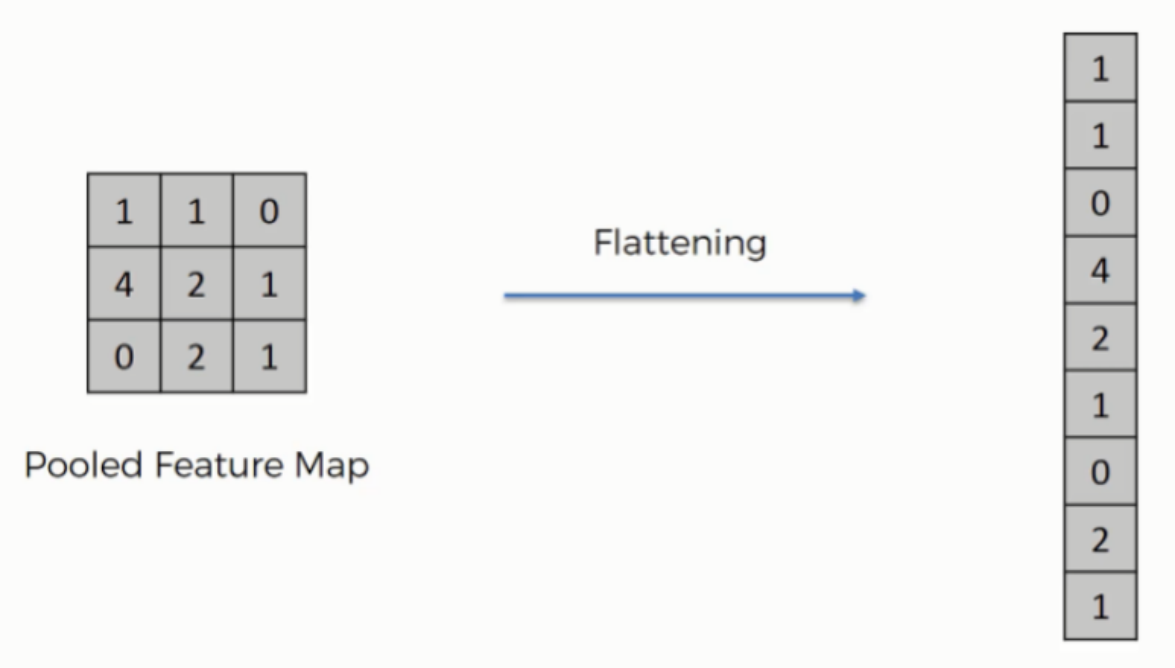
```
>>> import torch
>>> import torch

>>> channels = 3
>>> height, width = 6, 6
>>> x = torch.randn([channels, height, width])
>>> pool1 = torch.nn.MaxPool2d(
>>>     kernel_size=2, stride=2,
>>> )
>>> print(pool1(x).shape)
[3, 3, 3]  # Shape: [Channels, Height, Width]
```

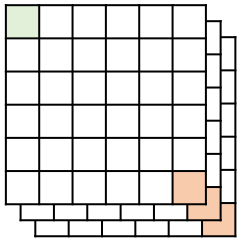
Commonly, `stride == kernel_size`

Flatten Layer (not explicitly shown sometimes)

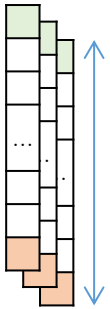
A flatten layer flattens the pooled feature map into a column, before feeding into the fully connect layer for classification:



PyTorch Flatten



Flatten



Height x Width

Usually, the top left pixel is the first, bottom right is the last

```
>>> import torch

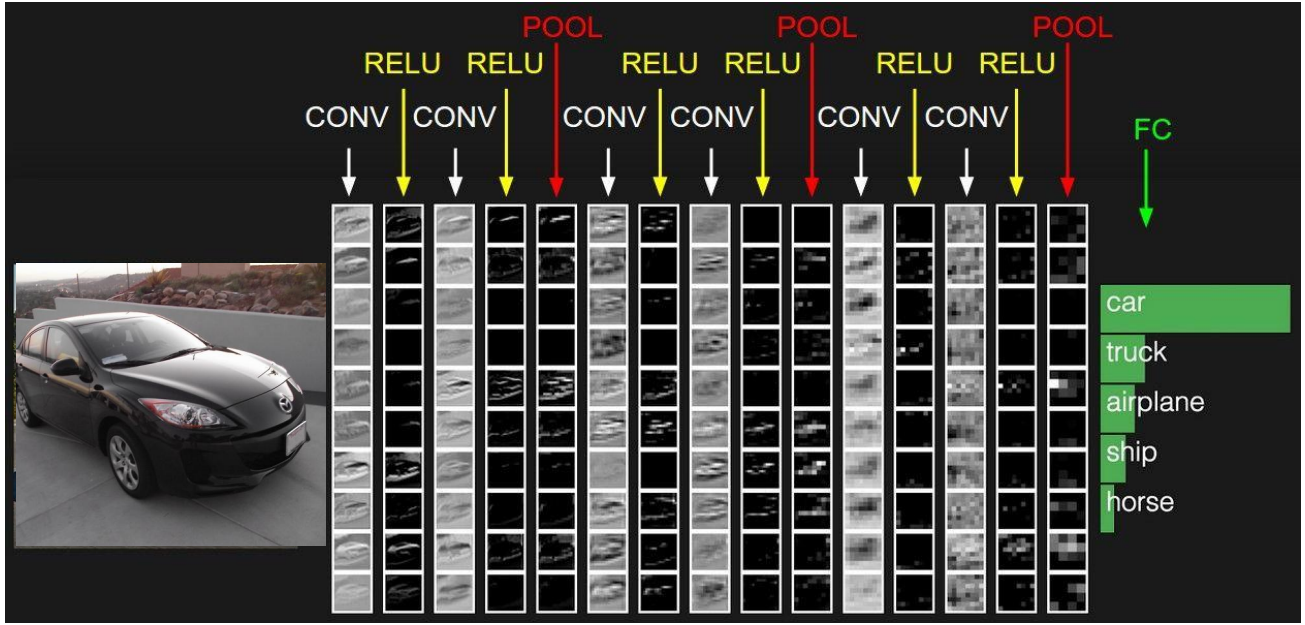
>>> channels = 2
>>> height, width = 6, 6
>>> x = torch.randn([channels, height, width])
>>> #                [    0   ,    1   ,   2   ]


>>> flat = torch.nn.Flatten(start_dim=1, end_dim=2)
>>> print(flat(x).shape)
[2, 36]
```

Flatten will squash dimensions of height **and** width into a combined dimension.

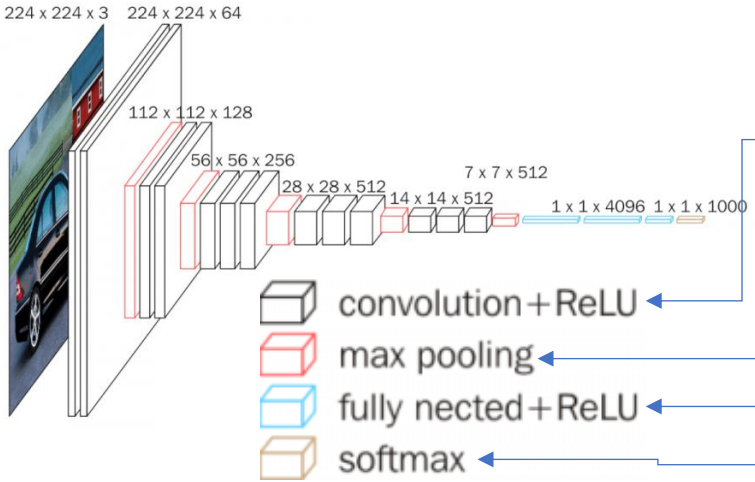
Fully Connected Layer (FC layer)

- Contains neurons that connect to the entire input volume, as in ordinary Neural Networks



PyTorch CNN Models implementation

CNNs are mostly implemented with the aforementioned layers.



`torch.nn.Conv2D + torch.nn.ReLU`

`torch.nn.MaxPool2D`

`torch.nn.Linear + torch.nn.ReLU`

`torch.nn.Softmax`

Always verify tensor shapes at each step to avoid dimension mismatch errors in PyTorch CNN implementations.

If Conv2D returns `[Channel, Height, Width]`, but Linear accepts `[Channel]`, how do we convert it? Can Linear accept `[Channel, 1, 1]`?

Summary

- ConvNets stack CONV,POOL,FC layers
- Trend towards smaller filters and deeper architectures
- Trend towards getting rid of POOL/FC layers (just CONV)
- Historically architectures looked like

  [(CONV-RELU)*N-POOL?]*M-(FC-RELU)*K,SOFTMAX

  where N is usually up to ~5, M is large, 0 <= K <= 2.
  - but recent advances such as ResNet/GoogLeNet have challenged this paradigm

CNN Python Tutorial

```
import tensorflow as tf

from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
```

Download and prepare the CIFAR10 dataset

The CIFAR10 dataset contains 60,000 color images in 10 classes, with 6,000 images in each class. The dataset is divided into 50,000 training images and 10,000 testing images. The classes are mutually exclusive and there is no overlap between them.

```
(train_images, train_labels), (test_images, test_labels) = datasets.cifar10.load_data()

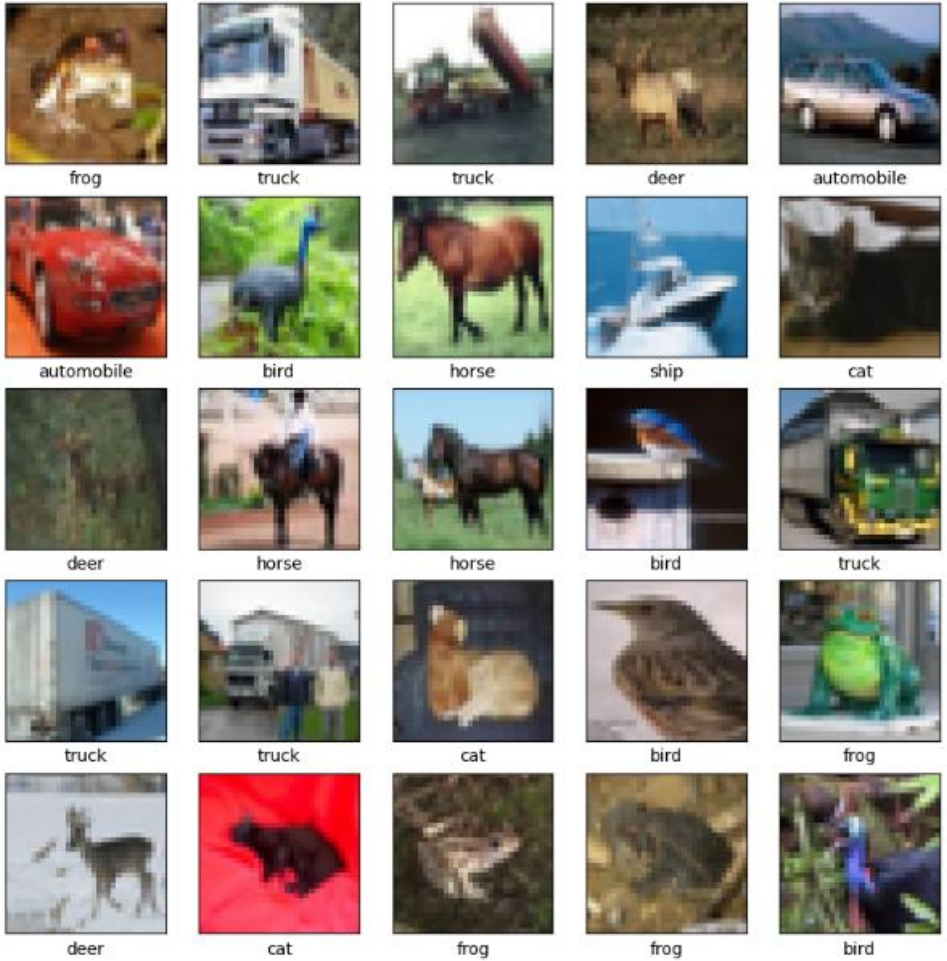
# Normalize pixel values to be between 0 and 1
train_images, test_images = train_images / 255.0, test_images / 255.0
```

Verify the data

To verify that the dataset looks correct, let's plot the first 25 images from the training set and display the class name below each image:

```
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i])
    # The CIFAR labels happen to be arrays,
    # which is why you need the extra index
    plt.xlabel(class_names[train_labels[i][0]])
plt.show()
```


frog
truck
truck
deer
automobile
automobile
bird
horse
ship
cat
deer
horse
horse
bird
truck
truck
truck
cat
bird
frog
deer
cat
frog
frog
bird

Create CNN

```
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
```

```
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10))
```

Here's the complete architecture of your model:

```
model.summary()
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 30, 30, 32)        896

 max_pooling2d (MaxPooling2  (None, 15, 15, 32)        0
 D)

 conv2d_1 (Conv2D)           (None, 13, 13, 64)        18496

 max_pooling2d_1 (MaxPoolin  (None, 6, 6, 64)          0
 g2D)

 conv2d_2 (Conv2D)           (None, 4, 4, 64)          36928

 flatten (Flatten)           (None, 1024)              0

 dense (Dense)               (None, 64)                65600

 dense_1 (Dense)             (None, 10)                650

=================================================================
Total params: 122570 (478.79 KB)
Trainable params: 122570 (478.79 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

The network summary shows that (4, 4, 64) outputs were flattened into vectors of shape (1024) before going through two Dense layers.

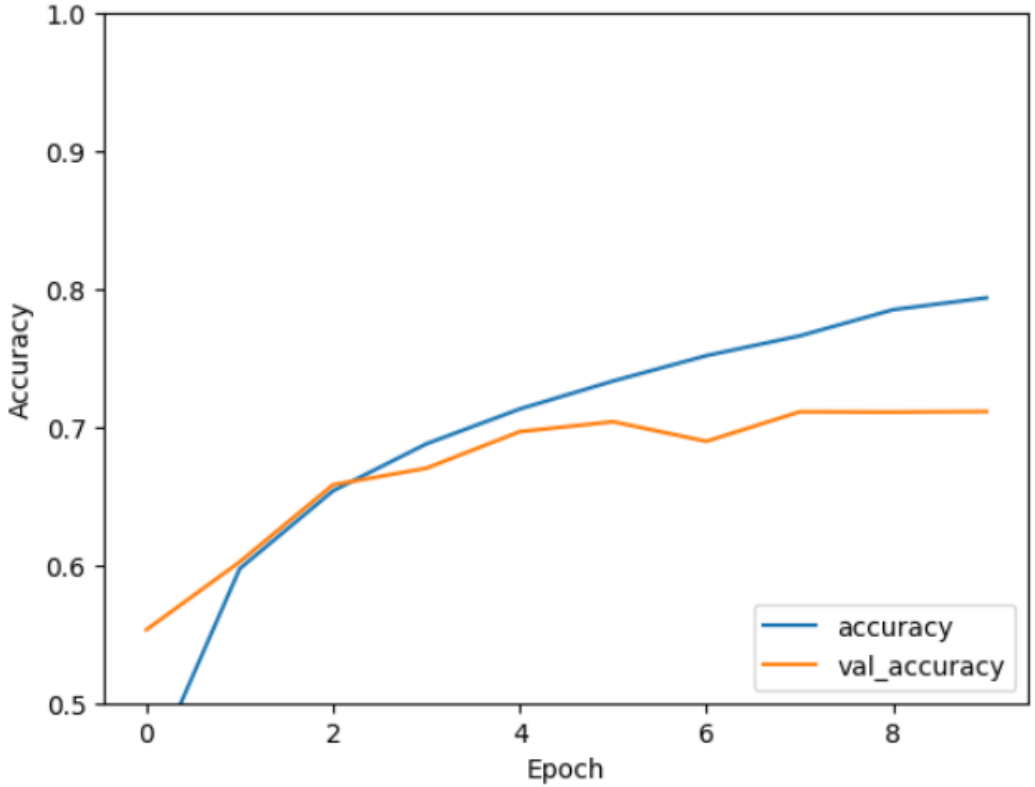
```
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

history = model.fit(train_images, train_labels, epochs=10,
                    validation_data=(test_images, test_labels))
```

Evaluate the model

```
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')

test_loss, test_acc = model.evaluate(test_images,  test_labels, verbose=2)
```




```
print(test_acc)
```

```
0.7113000154495239
```